1주차

데이터베이스 개론
의료데이터의 세계

- 데이터와 정보 — 혈압 수치는 언제 '진단'이 되는가
- 정형 · 반정형 · 비정형 의료데이터
- 병원 정보시스템 지도: EMR · LIS · PACS · CDW
- 한 학기 동안 함께할 데이터셋: 미니 EMR

> **이번 주 대표 질문**
> 혈압 120/80은 데이터인가, 정보인가?

이 수업이 끝나면 할 수 있는 것

- **임상 질문을 SQL로 번역한다**

  "당뇨 환자 중 HbA1c 검사를 안 받은 사람은?" → 컴퓨터가 알아듣는 문장으로

- **연구용 코호트를 직접 추출한다**

  포함·제외 기준을 SQL로 구현하고, 기초 특성표(Table 1)까지 산출한다

- **의료데이터의 함정을 알아본다**

  단위 불일치, 결측, 코드 버전 — "이 결과를 믿어도 되나?"에 답하기

용어	뜻
SQL	Structured Query Language — 관계형 데이터베이스에 질문을 던지는 국제 표준 언어
쿼리(query)	데이터베이스에 보내는 질의 한 건. "쿼리를 짠다" = 질문을 SQL 문장으로 쓴다
코호트(cohort)	공통 기준(진단·기간 등)으로 정의한 연구 대상 환자 집단
Table 1	논문의 첫 표 — 연구대상자의 기저 특성(성별·나이·동반질환) 요약

데이터 vs 정보 — 같은 숫자, 다른 무게

데이터 (Data)

- 관찰·측정·기록된 값
- `SBP 128, DBP 84` — 혈압계가 찍은 숫자
- 해석 기준은 별도로 적용된다
- 예: 검사 결과값, 내원 기록, 투약 기록

정보 (Information)

- 목적에 따라 데이터를 **가공·해석**한 것
- "고혈압전단계"(대한고혈압학회 2022 기준) — 기준이 붙은 판단
- 의사결정의 근거가 된다
- 예: 진료과별 평균 재원일수, 당화혈색소 조절률

> 이 수업에서 사용하는 실용적 구분입니다. 같은 128/84라도 **어떤 분류 기준을 적용하느냐**에 따라 범주가 달라집니다 — 기준과 맥락이 붙어야 데이터가 정보가 됩니다.

데이터 → 정보 → 의사결정: 병원에서의 실제 흐름

단계	예시	누가/무엇이 하나
데이터	P003 환자의 HbA1c = 8.9%	검사장비 → LIS 저장
정보	"최근 3회 연속 목표치 초과" (목표치 7.0%는 예시 — 실제 목표는 환자별로 다름)	SQL 질의로 추세 집계
지식	"추가 임상 평가가 필요한 환자군에 해당"	임상 지식 + 가이드라인
결정	치료 조정 여부 판단 — 다른 임상 정보와 함께 의료진이 결정	의료진

> 이 수업에서 배우는 SQL은 **2단계(데이터→정보)**를 담당합니다. LIS(검사정보시스템) 등 각 시스템 용어는 다음 슬라이드에서 정리합니다.

의료데이터의 세 가지 유형

구분	특징	병원에서의 예	저장되는 곳
정형 (Structured)	행·열 구조, 값의 형식이 고정	진단코드, 처방, 검사수치, 활력징후	관계형 DB (이 수업의 주인공)
반정형 (Semi-structured)	구조를 설명하는 태그 포함	HL7·FHIR(의료정보 교환 표준 규격)의 JSON·XML 메시지	문서형 DB, 파일 + 메타데이터
비정형 (Unstructured)	고정된 구조 없음	경과기록 자유서술, 영상 판독문, X-ray·MRI 영상, 심전도 파형	PACS, 전용 저장소

> 영상·파형·자유서술 기록처럼 관계형 테이블만으로 다루기 어려운 데이터가 병원에는 많습니다 — 14주차에 다시 다룹니다.

병원 정보시스템 지도 — 데이터는 왜 흩어져 있나

시스템	담는 것
EMR (전자의무기록)	진료기록, 진단, 경과기록
OCS (처방전달)	처방·오더의 발행과 전달
LIS (검사정보)	검체검사 결과, 정도관리
PACS (영상저장전송)	X-ray·CT·MRI 이미지와 판독문
CDW (임상데이터웨어하우스)	위 시스템들을 연구·분석용으로 통합

흩어져 있는 이유

- 시스템마다 **목적이 다르다** — 진료용 vs 검사용 vs 영상용
- 도입 시기·업체가 달라 **구조도 다르다**
- 많은 기관이 연구·분석용으로 **CDW나 별도 데이터마트**에 통합한다
- 이 수업의 미니 EMR은 이런 **분석용 통합 데이터셋**처럼 사용합니다

파일(엑셀)로 환자 명단을 관리하면 생기는 일

연구 데이터 관리에서 흔히 발생하는 일

- `환자명단_최종.xlsx`
- `환자명단_최종_수정.xlsx`
- `환자명단_최종_수정_진짜최종.xlsx`
- 세 파일의 환자 수가 **전부 다르다**
- 전화번호를 고쳤는데 **한 파일만** 고쳐졌다

이것이 '파일 시스템의 한계'

- **중복** — 같은 데이터가 여러 곳에 복사됨
- **불일치** — 하나만 수정되어 버전이 갈라짐
- **종속** — 파일 구조가 바뀌면 쓰던 수식·코드가 깨짐
- **동시성** — 두 사람이 같이 열면 한 명의 수정이 사라짐

> DBMS(데이터베이스 관리 시스템)는 중복·불일치·동시 접근 문제를 **통제하는 기능을 제공**합니다 — 데이터를 한 곳에 두고, 여러 사람이 동시에, 규칙을 지키며 쓰도록.

DBMS가 해주는 일 — 병원 버전

- **구조화된 저장**

  환자·내원·처방을 정해진 틀(테이블)에 — 적절한 설계와 결합하면 중복 최소화

- **규칙(제약조건) 강제**

  존재하지 않는 환자ID로는 처방을 입력할 수 없게

- **동시 접근 제어**

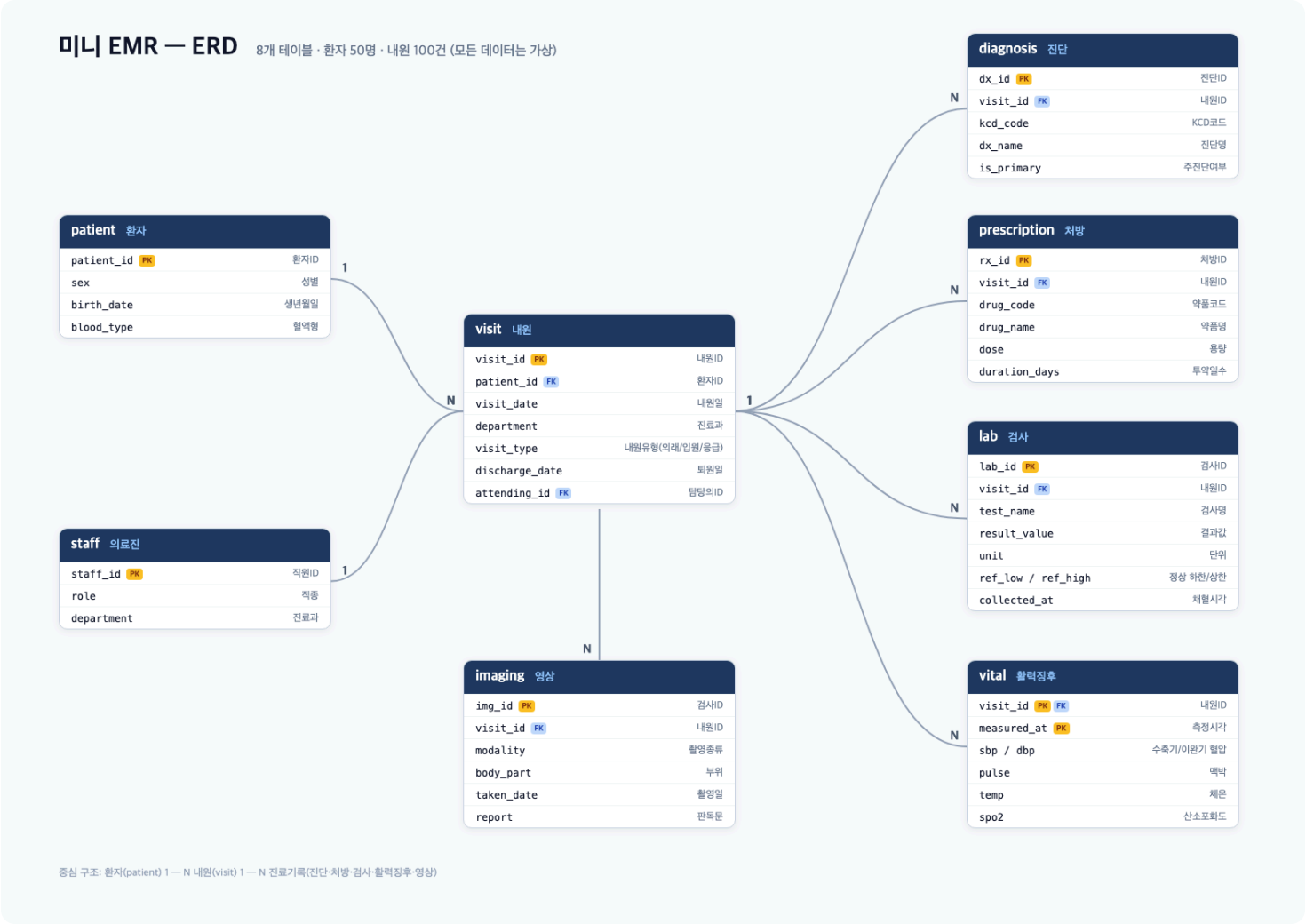
  외래 3진료실과 병동이 동시에 기록해도 안전하게

- **보안과 이력**

  권한 없는 접근 차단, 누가 언제 봤는지 기록

관계형 DBMS는 1970년 제안(E. F. Codd) 이후 표준으로 자리잡았고, 병원의 핵심 임상·원무 데이터는 여전히 **관계형 DB** 중심입니다.

한 학기 동안 함께할 데이터: 미니 EMR



가상 환자 50명, 내원 100건, 8개 테이블. 병원 데이터에서 흔히 나타나는 환자-내원-진료기록 관계를 단순화한 교육용 데이터셋입니다. **모든 데이터는 가상입니다.**

미니 EMR 8개 테이블 요약

테이블	내용	행 수	주요 컬럼
`patient` 환자	성별·생년월일·혈액형 (이름 없음 — 가명화)	50	patient_id
`visit` 내원	내원일·진료과·유형(외래/입원/응급)·퇴원일	100	visit_id, patient_id
`diagnosis` 진단	KCD코드·진단명·주진단여부	140	visit_id, kcd_code
`prescription` 처방	약품명·용량·투약일수	154	visit_id, drug_code
`lab` 검사	검사명·결과값·단위·정상범위·채혈시각	93	visit_id, test_name
`vital` 활력징후	혈압·맥박·체온·SpO2·측정시각	149	visit_id, measured_at
`staff` 의료진	직종·진료과	21	staff_id
`imaging` 영상	촬영종류·부위·판독문	21	visit_id, modality

> 왜 환자 이름이 없을까요? — 14주차(개인정보보호·가명처리)에서 다시 이야기합니다.

오늘의 첫 SQL — 실행해보기

실습 미리보기

임상 질문

미리보기: 우리 데이터셋의 환자 5명을 꺼내 봅시다

```
SELECT *
FROM patient
LIMIT 5;
```

patient_id	sex	birth_date	blood_type
P001	F	1947-04-12	B+
P002	F	1942-01-04	O+
P003	F	1942-12-27	A+
P004	M	1946-01-20	O+
P005	M	1950-05-09	AB+

총 5행

`SELECT` (무엇을 꺼낼지) · `FROM` (어느 테이블에서) · `LIMIT` (몇 행까지) — 정확한 문법은 3주차부터 다룹니다. 실습 환경은 브라우저(Colab)에서 링크 클릭만으로 열립니다.

실습 환경 — 설치 0

- **Google Colab 노트북**

  매주 링크 제공 → 클릭 → 바로 실행. 노트북·태블릿 모두 가능

- **SQLite**

  서버 없이 파일 하나로 동작하는 경량 관계형 DBMS — 기본 조회 문법은 다른 관계형 DBMS와 유사하나 세부 문법은 제품마다 다름

- **mini_emr.sqlite 다운로드**

  개인 PC에서는 DB Browser for SQLite(무료)로도 열람 가능

> 준비물: 구글 계정 하나면 충분합니다. 다음 주부터 매 수업 노트북을 함께 엽니다.

1주차 정리

- 데이터는 측정값, 정보는 **기준·맥락이 붙은 판단** — SQL은 그 변환 도구
- 병원 데이터는 EMR·LIS·PACS 등에 **흩어져** 있고, 연구는 CDW에서 시작된다
- 파일 관리의 중복·불일치 문제를 해결하는 것이 **DBMS**
- 실습은 **미니 EMR 하나로** 15주 전체 진행 — 8개 테이블 구조는 명세서(A4 배포물) 참조

다음 주 관계형 데이터베이스의 기본 구조 — 테이블, 키(Key), 그리고 "주민번호를 PK로 쓰면 안 되는 이유"

2주차

관계형 데이터베이스의 기본 구조
테이블 · 키 · 무결성 · NULL

- 테이블의 해부학: 튜플, 속성, 도메인
- 환자ID → 내원ID → 처방ID — 계층 구조와 1:N 관계
- 기본키(PK)와 외래키(FK), 그리고 주민번호 이야기
- NULL — "검사를 안 했다"와 "결과가 없다"는 다르다

> **이번 주 대표 질문**
> 주민등록번호를 환자 테이블의 기본키로 쓰면 왜 안 될까?

테이블의 구성 요소 — patient 테이블로 보기

용어

patient_id	sex	birth_date	blood_type
P001	F	1947-04-12	B+
P002	F	1942-01-04	O+
P003	F	1942-12-27	A+
… (총 50행)			

속성 Attribute 세로 열. 컬럼·필드라고도 부름 → 이 테이블은 4개 (**디그리** = 4)

튜플 Tuple 가로 행. 레코드라고도 부름 → 환자 한 명 = 한 행 (**카디널리티** = 50)

도메인 Domain 속성이 가질 수 있는 값의 범위 → sex는 {'M','F'}, blood_type은 8가지

관계형 테이블의 규칙 4가지 — 병원 기록으로 이해하기

- **튜플의 유일성**

  이론적 관계 모델은 중복 튜플을 허용하지 않는다 — 실제 SQL 테이블에서는 PK·UNIQUE 제약으로 관리

- **행 순서는 무의미**

  먼저 등록한 환자가 '위'에 있을 이유가 없다 — 순서는 조회할 때 정한다(ORDER BY)

- **속성의 원자성**

  한 칸에는 하나의 값 — '고혈압, 당뇨'를 한 칸에 넣는 순간 문제가 시작된다(10주차)

- **속성명의 고유성**

  한 테이블 안에 같은 이름의 컬럼은 없다

> 세 번째 규칙(원자성)을 어긴 실제 사례를 10주차 정규화에서 해부합니다.

병원 데이터의 계층: 환자 → 내원 → 진료기록

구조

환자 1명 (P010)

└ **내원 여러 번** (V0031, V0032, V0033…)

└ 내원마다 **진단 여러 개**

└ 내원마다 **처방 여러 건**

└ 내원마다 **검사 여러 건**

1 : N 관계

- 환자 1 : 내원 N
- 내원 1 : 진단 N / 처방 N / 검사 N / 활력징후 N
- 그래서 테이블을 **분리**해서 저장하고, 필요할 때 **연결(JOIN, 11주차)**한다
- 연결의 열쇠가 바로 **키(Key)**

키(Key)의 종류 — 미니 EMR에서 찾아보기

종류	정의	patient 테이블에서	visit 테이블에서
후보키	행을 유일하게 구분하는 최소 속성(조합)	patient_id	visit_id
기본키(PK)	후보키 중 대표로 선택된 키 NULL 불가 · 중복 불가	patient_id	visit_id
외래키(FK)	다른 테이블의 PK를 참조하는 키	—	patient_id (→ patient) attending_id (→ staff)
복합키	두 개 이상 속성을 합쳐야 유일해지는 키	—	vital: (visit_id, measured_at)

> 활력징후(vital)는 왜 복합키일까요? — 같은 내원에서 혈압을 **여러 번** 재기 때문에, '어느 내원의 몇 시 측정'까지 합쳐야 한 행이 특정됩니다 (이 교육용 데이터셋 기준 — 실무에서는 동시각 재측정 등을 고려해 대리키를 두기도 합니다).

주민등록번호를 PK로 쓰면 안 되는 이유

핵심

기술적 이유

- 바뀔 수 있다 — 정정·변경 사례 존재. PK가 바뀌면 **참조하는 모든 테이블**이 흔들린다
- 없을 수 있다 — 신생아, 외국인 환자, 응급 무명 환자
- 의미를 담고 있다 — 생년월일·성별이 노출되는 '스마트 키'는 위험하다

법적·윤리적 이유

- 개인정보보호법상 **고유식별정보** — 원칙적 처리 제한, 암호화 의무
- 유출 시 피해가 영구적 — 번호를 바꿔줄 수 없다
- 연구용 데이터는 **가명처리**가 기본(14주차)

그래서 병원은 업무상 의미가 없는 내부 식별자(등록번호, 우리 데이터의 P001…)를 PK로 씁니다 — **대리키(surrogate key)**. 법령 관련 내용은 개인정보보호법(2026-07 기준)의 교육용 요약이며, 실제 적용은 기관 지침을 확인하세요.

참조 무결성 — 잘못된 참조 입력의 차단

임상 질문

외래키가 지켜주는 것: 존재하지 않는 내원(V9999)에 처방을 넣으면?

```
INSERT INTO prescription
  (rx_id, visit_id, drug_code,
   drug_name, dose, duration_days)
VALUES
  ('R9999', 'V9999', 'MET500',
   '메트포르민', '500mg', 30);
```

실행 결과: **FOREIGN KEY constraint failed** — visit 테이블에 없는 V9999를 참조할 수 없습니다. **참조 무결성**: NULL이 아닌 외래키 값은 부모 테이블에 존재해야 한다. 기록에 없는 내원으로 처방이 입력되는 일을 DBMS가 차단합니다.

개체 무결성 vs 참조 무결성

개체 무결성 (PK의 규칙)

- 기본키는 **NULL 불가**, **중복 불가**
- 환자ID 없는 환자, 같은 ID의 두 환자는 존재할 수 없다
- 각 행은 기본키로 식별 가능해야 한다

참조 무결성 (FK의 규칙)

- NULL이 아닌 외래키 값은 **부모에 존재**해야 한다
- 없는 환자에게 내원 기록을 달 수 없다
- 참조값은 정의된 참조 규칙을 만족해야 한다

무결성(integrity): 데이터가 정해진 규칙을 위반한 상태로 존재하지 못하게 하는 성질. DBMS가 입력·수정 시점마다 자동 검사하며, 위반한 문장은 통째로 거부됩니다.

NULL — 병원 데이터에서 가장 오해받는 값

NULL

기록	의미	NULL인가?
Glucose = **0**	혈당이 0 (사실상 불가능한 측정값)	**아니오 — 0은 값이다**
Glucose = **NULL**	결과가 없다 (용혈로 재검 필요 등)	**예 — '모름/없음'**
Glucose 행 자체가 **없음**	검사를 아예 하지 않았다	NULL도 아니고 행이 없음
discharge_date = **NULL**	외래·응급 귀가 (입원이 아니었음)	구조적으로 해당 없음

"검사를 안 했다" ≠ "정상이었다". 연구에서 이 둘을 섞으면 유병률·이상률이 왜곡됩니다. NULL은 = 로 비교할 수 없고 `IS NULL` 로만 물을 수 있습니다(4주차).

NULL 만나보기 — 우리 데이터의 실제 사례

임상 질문

미니 EMR에 실제로 있는 NULL: 결과값이 비어 있는 검사를 찾아라

```
SELECT lab_id, test_name,
       result_value, unit,
       collected_at
FROM lab
WHERE result_value IS NULL;
```

lab_id	test_name	result_value	unit	collected_at
L0093	Creatinine	∅	mg/dL	2025-06-23 15:56:00

총 1행

L0093: 채혈은 했으나(collected_at 존재) 결과가 없습니다 — 검체 용혈로 재채혈이 필요했던 상황. 이 행을 지워버리면 '검사를 시도했다'는 사실 자체가 사라집니다.

스키마 — 구조를 먼저 약속한다

- **스키마(Schema)**

  테이블·컬럼·타입·제약조건의 설계도. 데이터보다 먼저 정의된다

- **외부/개념/내부의 3계층**

  사용자가 보는 화면 ↔ 병원 전체의 논리 구조 ↔ 디스크 저장 방식

- **데이터 독립성**

  저장 방식이 바뀌어도(디스크→클라우드) 쿼리는 그대로 — 계층 분리의 힘

> 7주차에 여러분이 직접 스키마를 정의(CREATE TABLE)합니다. 그때 오늘 배운 PK·FK·NULL 규칙이 전부 문법으로 다시 등장합니다.

이번 주 용어 정리

용어

용어	뜻
릴레이션(relation)	테이블의 정식 명칭 — 행(튜플)과 열(속성)의 2차원 구조
기본키(PK, Primary Key)	행을 유일하게 특정하는 대표 키 — NULL·중복 불가
외래키(FK, Foreign Key)	다른 테이블의 PK를 참조하는 컬럼 — 테이블 간 연결고리
복합키	두 개 이상 컬럼을 합쳐야 유일해지는 키 (예: vital의 내원ID+측정시각)
대리키(surrogate key)	의미 없는 내부 번호로 만든 PK (P001…) — 주민번호 대신 쓰는 이유
개체 무결성 / 참조 무결성	PK의 규칙(신원 확실) / FK의 규칙(연결 끊김 없음)
NULL	값이 없음(모름) — 0도, 빈칸도 아니며 =로 비교 불가
스키마(schema)	테이블·컬럼·타입·제약조건의 설계도

2주차 정리

- 테이블 = 튜플(행) × 속성(열), 병원 데이터는 **환자 1 : 내원 N**의 계층
- PK는 행의 신원(NULL·중복 불가), FK는 테이블 간 연결 — 두 무결성이 장부를 지킨다
- 주민번호는 PK로 쓰지 않는다 — **바뀌고, 없을 수 있고, 위험하다** → 의미 없는 대리키
- NULL은 0도 빈칸도 아니다 — **"안 했다"와 "정상"을 구분**하는 것이 의료데이터 해석의 기본

다음 주 SQL 시작 — SELECT로 데이터 꺼내기 ("우리 병원엔 어떤 진료과가 있나?")

3주차

SELECT ①
데이터 꺼내기의 기본기

- SELECT · FROM — 무엇을, 어디서
- 컬럼 고르기와 별칭(AS)
- DISTINCT — 중복 없이 보기
- ORDER BY · LIMIT — 정렬하고 자르기

> **이번 주 대표 질문**
> 우리 병원엔 어떤 진료과가 있나?

SQL 문장의 기본 골격

문법

```
SELECT 컬럼1, 컬럼2    -- 무엇을
FROM 테이블             -- 어디서
ORDER BY 컬럼           -- 어떤 순서로
LIMIT 숫자;             -- 몇 개만
```

- 영어 문장처럼 읽힌다: **"visit에서 내원일과 진료과를 골라줘"**
- 대소문자 구분 없음 (관례상 예약어는 대문자)
- 문장 끝에는 세미콜론 `;`
- `--` 뒤는 주석 — 실행되지 않는 메모

SELECT * — 일단 전부 꺼내기

임상 질문

환자 테이블 전체를 보여줘

```
SELECT *
FROM patient;
```

patient_id	sex	birth_date	blood_type
P001	F	1947-04-12	B+
P002	F	1942-01-04	O+
P003	F	1942-12-27	A+
P004	M	1946-01-20	O+
P005	M	1950-05-09	AB+
P006	F	1953-08-26	B+

… 총 50행 중 6행 표시

`*`는 '모든 컬럼'. 탐색 단계가 지나면 **필요한 컬럼만 지정**하고, 대용량 테이블은 조건이나 LIMIT 없이 조회하지 않는 것이 관례입니다.

컬럼 골라 꺼내기

임상 질문

환자의 ID와 생년월일만 보고 싶다

```
SELECT patient_id, birth_date
FROM patient;
```

patient_id	birth_date
P001	1947-04-12
P002	1942-01-04
P003	1942-12-27
P004	1946-01-20
P005	1950-05-09
P006	1953-08-26

… 총 50행 중 6행 표시

필요한 컬럼만 명시하는 습관 = 개인정보 최소 수집의 습관. 연구에 혈액형이 필요 없다면 꺼내지 않는 것이 원칙입니다.

별칭(AS) — 보고서용 이름 붙이기

임상 질문

컬럼 이름을 읽기 좋게 바꿔서 보여줘

```
SELECT patient_id AS 환자번호,
       birth_date AS 생년월일,
       blood_type AS 혈액형
FROM patient;
```

환자번호	생년월일	혈액형
P001	1947-04-12	B+
P002	1942-01-04	O+
P003	1942-12-27	A+
P004	1946-01-20	O+
P005	1950-05-09	AB+

… 총 50행 중 5행 표시

AS는 **표시 이름만** 바꿉니다. 원본 테이블은 그대로 — SELECT는 데이터를 절대 바꾸지 않는, 안전한 '읽기 전용' 명령입니다.

DISTINCT — 중복 없이 목록 보기

대표 질문

임상 질문

우리 병원엔 어떤 진료과가 있나?

```sql
SELECT DISTINCT department
FROM visit;
```

department
내과
가정의학과
신경과
외과
정형외과
응급의학과
소아청소년과

총 7행

visit 100행에는 진료과가 반복 등장하지만, DISTINCT가 중복을 걷어내고 **종류만** 보여줍니다. "우리 데이터에 어떤 값들이 들어있지?"를 묻는 탐색의 첫 도구.

DISTINCT 연습 — staff 테이블

임상 질문

의료진은 어떤 직종으로 구성되어 있나?

```
SELECT DISTINCT role
FROM staff;
```

role
의사
간호사
약사
임상병리사
방사선사

총 5행

새 데이터셋을 받으면 가장 먼저 할 일: 주요 범주형 컬럼마다 DISTINCT로 **값의 종류**를 훑어보기. 오타·코드 혼용이 여기서 발견됩니다.

ORDER BY — 정렬

임상 질문

환자를 나이 많은 순서로 정렬해줘

```
SELECT patient_id, birth_date, sex
FROM patient
ORDER BY birth_date ASC;
```

patient_id	birth_date	sex
P008	1938-03-09	F
P014	1939-09-26	F
P015	1940-10-21	M
P007	1941-09-27	F
P002	1942-01-04	F
P003	1942-12-27	F

… 총 50행 중 6행 표시

생년월일 **오름차순(ASC)** = 나이 **많은** 순. 헷갈리기 쉬운 지점입니다. 내림차순은 `DESC`. ASC는 기본값이라 생략 가능.

ORDER BY + LIMIT — 최신순 상위 N개

임상 질문

가장 최근 내원 5건은?

```sql
SELECT visit_id, patient_id,
       visit_date, department
FROM visit
ORDER BY visit_date DESC
LIMIT 5;
```

visit_id	patient_id	visit_date	department
V0093	P043	2025-12-06	응급의학과
V0001	P001	2025-11-22	내과
V0022	P007	2025-11-10	내과
V0040	P014	2025-11-10	내과
V0098	P047	2025-10-09	가정의학과

총 5행

정렬 없이 LIMIT만 쓰면 '아무 5건'이 나옵니다 — 행에는 순서가 없으니까요(2주차). '최근 5건'은 반드시 ORDER BY와 세트.

다중 정렬

임상 질문

정렬 기준이 두 개라면? — 진료과별로 묶고, 그 안에서 최신순

```
SELECT department, visit_date, visit_id
FROM visit
ORDER BY department ASC,
         visit_date DESC
LIMIT 8;
```

department	visit_date	visit_id
가정의학과	2025-10-09	V0098
가정의학과	2025-09-08	V0043
가정의학과	2025-07-06	V0049
가정의학과	2025-04-04	V0100
가정의학과	2025-03-26	V0027
가정의학과	2025-01-03	V0050
가정의학과	2024-11-17	V0026
가정의학과	2024-05-18	V0099

총 8행

쉼표 순서가 곧 우선순위입니다.

연습문제 — 직접 번역해 보세요

실습

- **Q1.** 검사(lab) 테이블에는 어떤 검사 종류들이 있는가?

  힌트: DISTINCT

- **Q2.** 처방(prescription)에서 약품명과 용량만, 약품명 가나다순으로

  힌트: ORDER BY

- **Q3.** 가장 어린 환자 3명의 ID와 생년월일은?

  힌트: birth_date DESC + LIMIT

- **Q4.** 영상(imaging) 테이블의 촬영종류 목록은?

> 정답 쿼리는 이번 주 Colab 노트북 마지막 '정답 및 해설' 섹션에 있습니다.

이번 주 용어 정리

용어

용어	뜻
예약어(keyword)	SELECT·FROM처럼 SQL이 미리 정해둔 단어 — 관례상 대문자로 표기
절(clause)	SELECT 절, FROM 절처럼 문장을 구성하는 블록 단위
별칭(alias)	AS로 붙이는 표시용 이름 — 원본 데이터는 바뀌지 않음
DISTINCT	결과에서 중복 행을 제거 — 값의 종류를 볼 때
ASC / DESC	오름차순(작→큰, 기본값) / 내림차순(큰→작)
LIMIT	반환할 행 수를 제한 — 큰 테이블 탐색의 안전장치
주석(comment)	'--' 뒤의 실행되지 않는 메모 — 쿼리의 의도를 기록

3주차 정리

- `SELECT 컬럼 FROM 테이블` — "무엇을, 어디서"가 모든 조회의 뼈대
- AS로 보고서용 이름, DISTINCT로 **값의 종류 탐색**
- ORDER BY 없는 LIMIT은 '아무거나 N개' — **최신순·상위권은 반드시 정렬과 세트**
- SELECT는 데이터를 변경하지 않는다 — 단, 대용량 조회의 실행 비용은 별개 문제

과제 Colab 노트북 3주차 연습문제 Q1~Q4 제출

다음 주 WHERE — 조건으로 골라내기 ("65세 이상 여성 환자는?", "I로 시작하는 진단은?")

4주차

SELECT ②
WHERE — 조건으로 골라내기

- 비교 연산자와 날짜 조건
- AND / OR / NOT — 조건 조합하기
- BETWEEN · IN · LIKE — 범위, 목록, 패턴
- IS NULL — 비어 있음을 묻는 법

이번 주 대표 질문

65세 이상 여성 환자는 몇 명인가?

WHERE — 행을 거르는 조건문

임상 질문

여성 환자만 보여줘

```sql
SELECT patient_id, sex, birth_date
FROM patient
WHERE sex = 'F';
```

patient_id	sex	birth_date
P001	F	1947-04-12
P002	F	1942-01-04
P003	F	1942-12-27
P006	F	1953-08-26
P007	F	1941-09-27
P008	F	1938-03-09

… 총 28행 중 6행 표시

문자값은 **작은따옴표** 'F'. WHERE는 조건이 참인 행만 통과시킵니다. 비교 연산자: `= != > < >= <=`

날짜 조건 — 문자열처럼 비교한다

임상 질문

1950년 이전에 태어난 환자는?

```sql
SELECT patient_id, birth_date
FROM patient
WHERE birth_date < '1950-01-01'
ORDER BY birth_date;
```

patient_id	birth_date
P008	1938-03-09
P014	1939-09-26
P015	1940-10-21
P007	1941-09-27
P002	1942-01-04
P003	1942-12-27

… 총 12행 중 6행 표시

날짜를 'YYYY-MM-DD' 형식으로 저장하면 문자열 비교가 곧 시간 순서가 됩니다. **형식 통일이 무너지면**(01/12/2024 vs 2024-12-01) 이 비교 전체가 무너집니다 — 14주차 주제.

AND — 두 조건 모두 만족

대표 질문

임상 질문

65세 이상 여성 환자는? (2026년 기준 연 나이)

```
SELECT patient_id, sex, birth_date,
       2026 - CAST(substr(birth_date,1,4) AS INT) AS
age
FROM patient
WHERE sex = 'F'
  AND 2026 - CAST(substr(birth_date,1,4) AS INT) >=
65;
```

patient_id	sex	birth_date	age
P001	F	1947-04-12	79
P002	F	1942-01-04	84
P003	F	1942-12-27	84
P006	F	1953-08-26	73
P007	F	1941-09-27	85
P008	F	1938-03-09	88

… 총 14행 중 6행 표시

substr으로 출생연도를 잘라 연 나이를 계산했습니다(간단하지만 생일 미도래를 무시하는 '연 나이'라는 한계 있음). 연구에서는 **어느 시점의 나이인가**(index date 기준)가 늘 쟁점입니다 — 13주차에서 다시.

OR — 둘 중 하나라도

임상 질문

응급 또는 입원으로 온 내원은?

```
SELECT visit_id, visit_date,
       department, visit_type
FROM visit
WHERE visit_type = '응급'
   OR visit_type = '입원'
ORDER BY visit_date DESC;
```

visit_id	visit_date	department	visit_type
V0093	2025-12-06	응급의학과	응급
V0085	2025-08-27	내과	입원
V0081	2025-08-10	내과	입원
V0088	2025-08-03	외과	입원
V0097	2025-05-05	소아청소년과	응급
V0079	2025-05-01	신경과	입원

… 총 29행 중 6행 표시

AND와 OR을 섞을 때는 **괄호**로 의도를 명시하세요: `WHERE (A OR B) AND C` 와 `WHERE A OR (B AND C)` 는 완전히 다른 환자군입니다.

IN — 목록 안에 있는가

임상 질문

같은 질문, 더 깔끔하게 — IN

```sql
SELECT visit_id, visit_date,
       department, visit_type
FROM visit
WHERE visit_type IN ('응급', '입원')
ORDER BY visit_date DESC;
```

visit_id	visit_date	department	visit_type
V0093	2025-12-06	응급의학과	응급
V0085	2025-08-27	내과	입원
V0081	2025-08-10	내과	입원
V0088	2025-08-03	외과	입원
V0097	2025-05-05	소아청소년과	응급
V0079	2025-05-01	신경과	입원

… 총 29행 중 6행 표시

OR을 여러 번 쓰는 대신 IN (값1, 값2, …). 진단코드 목록, 약물 목록을 다룰 때 필수 도구가 됩니다. 반대는 `NOT IN`.

BETWEEN — 범위 조건 (양 끝 포함)

임상 질문

2024년에 발생한 내원만 보려면?

```
SELECT visit_id, visit_date, department
FROM visit
WHERE visit_date BETWEEN '2024-01-01'
                     AND '2024-12-31'
ORDER BY visit_date;
```

visit_id	visit_date	department
V0091	2024-01-01	응급의학과
V0042	2024-01-11	가정의학과
V0031	2024-01-15	내과
V0089	2024-01-15	정형외과
V0032	2024-01-26	내과
V0029	2024-02-04	내과

… 총 28행 중 6행 표시

'2024-12-31' 내원도 포함됩니다(양끝 포함) — 연구 기간 정의의 기본 문형.

LIKE — 패턴으로 찾기

대표 질문

임상 질문

순환기계 질환(KCD I코드) 진단을 모두 찾아라

```
SELECT dx_id, kcd_code, dx_name
FROM diagnosis
WHERE kcd_code LIKE 'I%'
ORDER BY kcd_code;
```

dx_id	kcd_code	dx_name
D0001	I10	본태성 고혈압
D0003	I10	본태성 고혈압
D0005	I10	본태성 고혈압
D0007	I10	본태성 고혈압
D0008	I10	본태성 고혈압
D0015	I10	본태성 고혈압
D0017	I10	본태성 고혈압

… 총 73행 중 7행 표시

`%`는 '아무 글자 여러 개', `_`는 '아무 글자 한 개'. KCD는 알파벳 첫 글자가 질환 대분류라서 **LIKE 'I%' = 순환기계 전체**가 됩니다. 코드 체계를 알면 LIKE 한 줄이 강력한 필터가 됩니다.

LIKE 연습 — 코드 패밀리 찾기

임상 질문

당뇨(E11) 계열 진단만 — 더 구체적인 패턴

```
SELECT DISTINCT kcd_code, dx_name
FROM diagnosis
WHERE kcd_code LIKE 'E11%';
```

kcd_code	dx_name
E11.9	2형 당뇨병

총 1행

E11.0, E11.9처럼 소수점 세분류가 붙어도 'E11%'가 전부 잡습니다. 연구 정의서에서 '당뇨병: E11.x'라고 쓰는 것이 이 패턴입니다.

IS NULL — 비어 있음을 묻는 유일한 방법

임상 질문

아직 퇴원일이 기록되지 않은 입원 vs 외래·응급 — NULL 구분하기

```
SELECT visit_id, visit_type,
       visit_date, discharge_date
FROM visit
WHERE discharge_date IS NULL
ORDER BY visit_type;
```

visit_id	visit_type	visit_date	discharge_date
V0001	외래	2025-11-22	∅
V0002	외래	2024-08-16	∅
V0003	외래	2023-03-23	∅
V0004	외래	2024-10-13	∅
V0005	외래	2025-05-23	∅
V0006	외래	2025-07-21	∅

… 총 79행 중 6행 표시

discharge_date = NULL 은 아무 행도 찾지 못합니다 — NULL은 '='로 비교 불가(2주차). 반드시 `IS NULL / IS NOT NULL`. 이 데이터에서 퇴원일 NULL은 외래·응급 귀가라는 **구조적 의미**입니다.

연습문제 — 조건을 조합해 보세요

실습

- **Q1.** 남성이면서 O+ 혈액형인 환자는?
- **Q2.** 2023년 이후 신경과 내원을 최신순으로
- **Q3.** 호흡기계(J코드) 진단을 모두 찾아라

  힌트: LIKE
- **Q4.** 투약일수가 30일 이상 90일 이하인 처방은?

  힌트: BETWEEN
- **Q5.** 정상 상한(ref_high)이 기록되지 않은 검사가 있는가?

  힌트: IS NULL

이번 주 용어 정리

용어	뜻
조건식(술어, predicate)	WHERE 뒤에서 행마다 참/거짓으로 판정되는 식
논리 연산자	AND(모두 만족) · OR(하나라도) · NOT(부정) — 섞을 땐 괄호로 우선순위 명시
BETWEEN A AND B	범위 조건 — 양쪽 끝 값 포함
IN (목록)	목록의 값 중 하나와 일치 — 코드 목록 필터의 기본
LIKE / 와일드카드	패턴 검색 — %(아무 글자 여러 개), _(아무 글자 한 개)
IS NULL / IS NOT NULL	값 없음/있음 검사 — '=NULL'은 아무 행도 찾지 못함
연 나이	올해 − 출생연도 (생일 미도래 무시) — 만 나이와 다르므로 논문엔 정의 명시
KCD	한국표준질병·사인분류 — WHO ICD 기반 국내 진단코드 체계, 현재 8차 개정(KCD-8)

4주차 정리

- WHERE = 행 필터. 비교 연산자 + **AND/OR(괄호로 명시)**로 조건을 쌓는다
- IN(목록) · BETWEEN(범위, 양끝 포함) · LIKE(패턴) — **연구 정의서의 3대 문형**
- KCD 코드 체계 × LIKE = 질환군 필터 (`LIKE 'I%'` → 순환기계)
- NULL은 `IS NULL`로만 — **=NULL은 조용히 실패한다**

과제 Colab 4주차 Q1~Q5 + '내 연구 관심 질환의 KCD 패턴' 한 줄 찾아오기

다음 주 계산과 집계 — 재원일수 구하기, CASE로 혈압 분류하기, COUNT/AVG

5주차

SELECT ③
계산하고, 분류하고, 요약하기

- 산술식 — 재원일수 계산
- CASE — 혈압을 정상 · 주의 · 고혈압으로
- 집계함수 — COUNT · AVG · MIN · MAX
- NULL이 집계를 어떻게 왜곡하는가

> **이번 주 대표 질문**
> 우리 병원의 평균 재원일수는 며칠인가?

산술식 — 컬럼으로 계산하기

임상 질문

입원 환자들은 며칠씩 입원했나? — 재원일수 계산

```
SELECT visit_id, visit_date, discharge_date,
       julianday(discharge_date)
         - julianday(visit_date) AS los
FROM visit
WHERE visit_type = '입원'
ORDER BY los DESC;
```

visit_id	visit_date	discharge_date	los
V0089	2024-01-15	2024-02-19	35
V0073	2023-03-28	2023-04-17	20
V0061	2020-12-13	2021-01-01	19
V0069	2023-01-31	2023-02-19	19
V0077	2022-12-14	2022-12-31	17
V0075	2021-05-25	2021-06-08	14

… 총 21행 중 6행 표시

SELECT 안에서 사칙연산이 가능합니다. SQLite에서 날짜 간격은 `julianday()` 차이로 구합니다(일 단위). LOS(Length of Stay, 재원일수)는 병원 운영·연구의 기본 지표.

산술식 + ROUND — 비율 계산

임상 질문

검사 결과가 정상 상한을 몇 배나 초과했나?

```
SELECT test_name, result_value, ref_high,
       ROUND(result_value / ref_high, 2)
         AS ratio
FROM lab
WHERE test_name = 'CRP'
ORDER BY ratio DESC;
```

test_name	result_value	ref_high	ratio
CRP	169	5	33.8
CRP	128.4	5	25.68
CRP	109.7	5	21.94
CRP	105	5	21
CRP	54.7	5	10.94

… 총 6행 중 5행 표시

CRP가 정상 상한(5)의 최대 30배가 넘는 환자들 — 폐렴 입원군입니다. ROUND(값, 자릿수)로 소수점 정리.

CASE — SQL의 조건 분기

문법

```
CASE
  WHEN 조건1 THEN 결과1
  WHEN 조건2 THEN 결과2
  ELSE 나머지결과
END
```

임상에서 늘 하는 일

- 수치 → **범주**: 혈압 → 정상/주의/고혈압
- 나이 → 연령대: 65세 기준 고령 여부
- 검사값 → 정상/이상 플래그
- 즉, **가이드라인의 분류표를 코드로** 옮기는 문법

CASE 실전 — 혈압 분류

대표 질문

임상 질문

활력징후의 혈압을 정상 · 주의 · 고혈압으로 분류하라

```sql
SELECT visit_id, sbp, dbp,
  CASE
    WHEN sbp >= 140 OR dbp >= 90  THEN '고혈압'
    WHEN sbp >= 120               THEN '주의'
    ELSE '정상'
  END AS bp_grade
FROM vital
ORDER BY sbp DESC;
```

visit_id	sbp	dbp	bp_grade
V0005	163	116	고혈압
V0016	163	128	고혈압
V0033	163	127	고혈압
V0025	162	122	고혈압
V0045	162	113	고혈압
V0024	160	115	고혈압

… 총 149행 중 6행 표시

WHEN은 **위에서부터 차례로** 평가되고 먼저 맞은 조건에서 멈춥니다 — 분류 기준의 순서가 곧 로직입니다. 분류 기준(140/90)을 바꾸면 연구 결과도 바뀝니다: 기준 명시가 왜 중요한지 보이는 대목.

집계함수 — 여러 행을 하나의 숫자로

임상 질문

우리 병원 전체 요약: 내원 건수, 평균 재원일수, 최장 입원

```sql
SELECT COUNT(*) AS 입원건수,
       ROUND(AVG(julianday(discharge_date)
         - julianday(visit_date)), 1) AS 평균재원일,
       MAX(julianday(discharge_date)
         - julianday(visit_date)) AS 최장재원일
FROM visit
WHERE visit_type = '입원';
```

입원건수	평균재원일	최장재원일
21	11.7	35

총 1행

COUNT·SUM·AVG·MIN·MAX. 집계함수가 등장하면 결과는 **요약된 한 행**이 됩니다. 최장 35일 입원(대퇴골 골절 환자)이 평균을 끌어올리고 있죠 — 평균만 보면 놓치는 것들.

CASE × 집계 — 비율 구하기

대표 질문

임상 질문

검사 결과 중 정상범위를 벗어난 비율은 몇 %인가?

```
SELECT COUNT(*) AS 전체검사,
  SUM(CASE WHEN result_value > ref_high
            OR result_value < ref_low
           THEN 1 ELSE 0 END) AS 이상건수,
  ROUND(100.0 * SUM(CASE WHEN result_value > ref_high
            OR result_value < ref_low
           THEN 1 ELSE 0 END) / COUNT(*), 1)
    AS 이상비율
FROM lab;
```

전체검사	이상건수	이상비율
93	79	84.9

총 1행

CASE로 1/0 플래그를 만들고 SUM으로 세는 패턴 — 유병률, 달성률, 이상률 계산의 기본 조합입니다. 100.0의 **.0**이 없으면 정수 나눗셈이 되어 0이 나옵니다.

NULL과 집계 — 조용한 왜곡

임상 질문

COUNT(*) vs COUNT(컬럼) — NULL이 끼어들면?

```
SELECT COUNT(*) AS 전체행수,
       COUNT(result_value) AS 값있는행수,
       COUNT(*) - COUNT(result_value)
         AS 결측행수
FROM lab;
```

전체행수	값있는행수	결측행수
93	92	1

총 1행

`COUNT(*)` 는 행 수, `COUNT(컬럼)` 은 **NULL을 제외한** 값의 수. AVG도 NULL을 빼고 평균합니다 — 결측이 많은 검사의 '평균'은 **검사받은 사람들만의 평균**임을 항상 기억하세요.

MIN/MAX — 데이터 범위 점검

임상 질문

체온과 산소포화도의 분포 감 잡기

```
SELECT MIN(temp) AS 최저체온,
       MAX(temp) AS 최고체온,
       ROUND(AVG(temp), 1) AS 평균체온,
       MIN(spo2) AS 최저SpO2
FROM vital;
```

최저체온	최고체온	평균체온	최저Sp02
36.2	37.2	36.7	94

총 1행

새 데이터를 받으면 MIN/MAX부터: 체온 3.6℃, SpO2 250% 같은 **불가능한 값**이 여기서 걸립니다. 오늘 데이터는 깨끗하지만, 14주차에 지저분한 사례를 만납니다.

연습문제

실습

- **Q1.** 처방의 평균 투약일수는?
- **Q2.** 맥박을 서맥(<60) · 정상 · 빈맥(>100)으로 분류해 보라

  힌트: CASE
- **Q3.** HbA1c 검사의 최솟값·최댓값·평균은?

  WHERE test_name='HbA1c'
- **Q4.** HbA1c가 7.0 이상인 검사 건수의 비율은?

  힌트: CASE×SUM 패턴
- **Q5.** 입원 재원일수가 14일을 넘는 내원은 몇 건인가?

이번 주 용어 정리

용어

용어	뜻
산술식	SELECT 안의 사칙연산 — 컬럼끼리 계산해 새 값을 만든다
집계함수(aggregate)	여러 행을 값 하나로 요약 — COUNT·SUM·AVG·MIN·MAX
CASE 식	SQL의 조건 분기 — WHEN을 위에서부터 평가, 먼저 맞으면 멈춤
플래그(flag)	CASE로 만든 1/0 표시값 — SUM하면 개수, 평균 내면 비율
LOS (Length of Stay)	재원일수 — 퇴원일과 내원일의 차이
julianday()	날짜를 일 단위 숫자로 바꾸는 SQLite 함수 — 날짜 간격 계산용
결측(missing)	값이 없는 상태 — AVG·COUNT(컬럼)는 NULL을 빼고 계산

5주차 정리

- SELECT 안에서 계산 가능 — 재원일수 = `julianday()` 차이
- CASE = 가이드라인 분류표의 코드화. **WHEN의 순서가 로직**
- **CASE(1/0) × SUM ÷ COUNT** — 임상 비율 지표의 기본 패턴 (100.0 잊지 말기)
- AVG·COUNT는 NULL을 조용히 제외한다 — **'누구의 평균인가'를 항상 물을 것**

과제 Colab 5주차 Q1~Q5

다음 주 GROUP BY — 진료과별, 연령대별로 묶어 세기. Table 1의 원형

6주차

SELECT ④
GROUP BY — 묶어서 요약하기

- GROUP BY — 그룹별 집계의 원리
- 진료과별 · 유형별 · 월별로 세기
- HAVING — 그룹에 거는 조건
- Table 1 맛보기 — 논문 첫 표의 정체

> **이번 주 대표 질문**
> 진료과별 환자 수는 각각 몇 명인가?

GROUP BY의 원리 — 나눠 담고, 각각 요약

원리

GROUP BY가 하는 일

① 지정한 컬럼의 값이 같은 행끼리 **바구니에 나눠 담고**

② 바구니마다 집계함수를 **한 번씩** 적용

③ 바구니 수만큼의 행을 반환

철칙 하나

SELECT에는 ① **GROUP BY에 쓴 컬럼**과 ② **집계함수**만 올 수 있습니다.

그룹당 한 행으로 접히는데, 그룹 안에서 값이 제각각인 일반 컬럼은 '어느 값을 보여줄지' 정할 수 없기 때문.

GROUP BY 첫 실전

대표 질문

임상 질문

진료과별 내원 건수와 환자 수는?

```sql
SELECT department,
       COUNT(*) AS 내원건수,
       COUNT(DISTINCT patient_id) AS 환자수
FROM visit
GROUP BY department
ORDER BY 내원건수 DESC;
```

department	내원건수	환자수
내과	57	23
신경과	22	10
가정의학과	10	6
응급의학과	7	7
외과	2	2
정형외과	1	1
소아청소년과	1	1

총 7행

내원건수 ≠ 환자수에 주목 — 만성질환 외래는 같은 환자가 반복 방문합니다. `COUNT(DISTINCT patient_id)`가 '사람 수'입니다. 행 단위(내원)와 사람 단위를 혼동하면 유병률이 뻥튀기됩니다.

그룹별 다중 집계

임상 질문

내원 유형(외래/입원/응급)별로 평균 재원일수까지

```
SELECT visit_type,
       COUNT(*) AS 건수,
       ROUND(AVG(julianday(discharge_date)
         - julianday(visit_date)), 1)
         AS 평균재원일
FROM visit
GROUP BY visit_type;
```

visit_type	건수	평균재원일
외래	71	∅
응급	8	∅
입원	21	11.7

총 3행

외래·응급의 평균재원일은 NULL — discharge_date가 없으니 AVG가 계산할 대상이 없습니다. 지난주의 'NULL은 조용히 빠진다'가 그룹별로도 그대로 적용.

날짜 함수 × GROUP BY

임상 질문

월별 내원 추이 — 시간 축으로 묶기

```sql
SELECT strftime('%Y-%m', visit_date) AS 월,
       COUNT(*) AS 내원건수
FROM visit
WHERE visit_date >= '2025-01-01'
GROUP BY strftime('%Y-%m', visit_date)
ORDER BY 월;
```

월	내원건수
2025-01	3
2025-02	1
2025-03	3
2025-04	2
2025-05	7
2025-06	4
2025-07	3

… 총 12행 중 7행 표시

`strftime('%Y-%m', …)`으로 '연-월'만 뽑아 묶으면 월별 통계. 감염병 유행 곡선, 계절성 분석이 전부 이 문형입니다.

HAVING — 그룹에 거는 조건

대표 질문

임상 질문

10건 이상 처방된 약물만 보여줘

```
SELECT drug_name,
       COUNT(*) AS 처방건수
FROM prescription
GROUP BY drug_name
HAVING COUNT(*) >= 10
ORDER BY 처방건수 DESC;
```

drug_name	처방건수
암로디핀	46
아토르바스타틴	25
메트포르민	24
클로피도그렐	20
아세트아미노펜	17

총 5행

WHERE는 **행**을 거르고(집계 전), HAVING은 **그룹**을 거릅니다(집계 후). '처방건수 10건 이상'은 집계가 끝나야 알 수 있으니 HAVING 자리입니다.

WHERE vs HAVING — 언제 어느 쪽?

WHERE (집계 전, 행 필터)

- "2024년 내원 중에서" — 원본 행 조건
- 집계함수를 **쓸 수 없다**
- 먼저 실행되어 계산량을 줄인다

HAVING (집계 후, 그룹 필터)

- "환자 수가 5명 이상인 진료과만" — 요약값 조건
- 집계함수를 **쓰기 위한** 자리
- GROUP BY 다음에 실행된다

실행 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. "2024년(WHERE) 내원 기준으로, 내원이 10건 이상인(HAVING) 진료과"처럼 둘을 함께 쓰는 게 보통입니다.

다중 컬럼 GROUP BY — Table 1 맛보기

Table 1

임상 질문

성별 × 연령대 분포 — Table 1의 첫 줄 만들기

```
SELECT sex,
  CASE WHEN 2026 - CAST(substr(birth_date,1,4) AS
INT) >= 65
       THEN '65세 이상' ELSE '65세 미만'
  END AS 연령군,
  COUNT(*) AS 명수
FROM patient
GROUP BY sex, 연령군
ORDER BY sex, 연령군;
```

sex	연령군	명수
F	65세 미만	14
F	65세 이상	14
M	65세 미만	16
M	65세 이상	6

총 4행

논문의 Table 1(연구대상자 특성표)은 결국 **CASE로 범주를 만들고 GROUP BY로 세는 것**의 반복입니다. 13주차에 코호트 전용 Table 1을 완성합니다.

지난 3주의 총결산 — 필터 × CASE × 집계 × 그룹

임상 질문

진료과별로 '고혈압(sbp≥140) 측정 비율'까지 — 종합 연습

```sql
SELECT v.department,
       COUNT(*) AS 측정수,
       ROUND(100.0 * SUM(CASE WHEN t.sbp >= 140
         THEN 1 ELSE 0 END) / COUNT(*), 1)
         AS 고혈압비율
FROM vital t, visit v
WHERE t.visit_id = v.visit_id
GROUP BY v.department
ORDER BY 고혈압비율 DESC;
```

department	측정수	고혈압비율
신경과	46	100
내과	69	21.7
가정의학과	10	10
정형외과	5	0
응급의학과	14	0
외과	4	0
소아청소년과	1	0

총 7행

`FROM vital t, visit v` + WHERE 연결은 **암시적 조인** — 11주차에 배울 `JOIN … ON`과 동작이 같습니다. WHERE·CASE·SUM·GROUP BY가 한 쿼리에 모두 들어간 형태로, 중간고사 최고 난도 수준입니다.

연습문제

실습

- **Q1.** KCD 코드별 진단 건수를 많은 순으로
- **Q2.** 검사 종류별 평균 결과값과 검사 건수는?
- **Q3.** 5회 이상 내원한 환자의 ID와 내원 횟수는?

  힌트: GROUP BY patient_id + HAVING
- **Q4.** 혈액형별 환자 수 분포는?
- **Q5.** 연도별 입원 건수 추이는?

  힌트: strftime('%Y', …)

중간고사 안내: 9주차 시험 범위는 1~8주차 전체. SELECT 4부작(3~6주차)이 배점의 중심입니다.

이번 주 용어 정리

용어

용어	뜻
GROUP BY	지정 컬럼의 값이 같은 행끼리 묶어 그룹당 한 행으로 요약
HAVING	집계가 끝난 그룹에 거는 조건 (WHERE는 집계 전 행에)
COUNT(DISTINCT x)	중복을 빼고 센 개수 — '내원 수'가 아닌 '사람 수'를 셀 때
strftime()	날짜에서 원하는 부분을 추출하는 SQLite 함수 — '%Y-%m'이면 연-월
실행 순서	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
암시적 조인	FROM a, b + WHERE 연결 — JOIN…ON(11주차)과 같은 동작의 옛 문법
Table 1	연구대상자 기저 특성표 — CASE로 범주를 만들고 GROUP BY로 세면 완성

6주차 정리

- GROUP BY = 같은 값끼리 담아 **그룹당 한 행**으로 요약
- **내원 수 ≠ 환자 수** — '사람 수'는 COUNT(DISTINCT patient_id)로 센다
- WHERE는 행(집계 전), HAVING은 그룹(집계 후) — 실행 순서가 근거
- CASE로 범주 만들고 GROUP BY로 세면 그것이 **Table 1**

과제 Colab 6주차 Q1~Q5

다음 주 CREATE TABLE — 이번엔 우리가 테이블을 만든다: 연구 데이터셋 설계

7주차

CREATE · ALTER · DROP
테이블 설계 = 연구 데이터셋 설계

- 데이터 타입 — 검사값은 왜 REAL인가
- CREATE TABLE과 제약조건 (NOT NULL · UNIQUE · CHECK · FK)
- ALTER · DROP — 구조 변경과 삭제
- 실습: 내 연구용 코호트 테이블 설계

> **이번 주 대표 질문**
> 내 연구의 증례기록지(CRF)를 테이블로 옮긴다면?

데이터 타입 — 값의 성질을 먼저 정한다

타입	담는 것	미니 EMR에서	타입을 잘못 고르면
INTEGER	정수	투약일수, 혈압, 맥박	—
REAL	소수	검사 결과값, 체온	TEXT로 넣으면 **평균·비교가 문자 기준**으로 왜곡
TEXT	문자열	진단명, 판독문, ID	숫자를 TEXT로: '9' > '10' 이 참이 되는 사고
DATE / DATETIME	날짜·시각	내원일, 채혈시각	형식 제각각이면 기간 필터 전멸 (14주차)

검사값을 REAL로 선언하는 순간 '결과값엔 숫자만'이라는 **규칙이 구조에 새겨집니다**. 타입은 첫 번째 데이터 검증 장치입니다.

CREATE TABLE — 미니 EMR의 실제 진단 테이블

DDL

```sql
CREATE TABLE diagnosis (
  dx_id      TEXT PRIMARY KEY,              -- 진단ID
  visit_id   TEXT NOT NULL
             REFERENCES visit(visit_id),    -- 어느 내원의 진단인가
  kcd_code   TEXT NOT NULL,                 -- 상병코드는 필수
  dx_name    TEXT NOT NULL,
  is_primary INTEGER NOT NULL
             CHECK (is_primary IN (0,1))    -- 주진단 여부는 0/1만
);
```

지금까지 조회해 온 diagnosis 테이블의 실제 정의문입니다. 구조(컬럼·타입) + 규칙(제약조건)을 한 번에 선언합니다.

제약조건 — 입력 사고를 구조로 막는다

제약조건	규칙	병원에서의 의미
PRIMARY KEY	NULL 불가 + 중복 불가	모든 기록엔 유일한 신원
NOT NULL	빈 값 금지	상병코드 없는 진단은 접수 자체가 안 되게
UNIQUE	중복 금지 (SQLite에서는 NULL 여러 개 허용 — DBMS마다 다름)	같은 장비 시리얼번호 이중 등록 방지
CHECK	조건을 만족하는 값만	성별은 M/F만, SpO2는 0~100만
FOREIGN KEY	NULL이 아니면 부모에 존재하는 값만	없는 환자·없는 내원에 기록 못 달게 (2주차)

제약조건(constraint): 컬럼·테이블에 선언하는 입력 규칙. 위반하면 'constraint failed' 에러와 함께 **문장 전체가 거부**되어 부분 입력이 남지 않습니다.

제약조건 위반 실험

임상 질문

CHECK가 일하는 순간: SpO2에 250을 입력해 본다면?

```
CREATE TABLE vital_demo (
  visit_id TEXT,
  spo2 INTEGER CHECK (spo2 BETWEEN 0 AND 100)
);
INSERT INTO vital_demo VALUES ('V0001', 250);
```

실행 결과: **CHECK constraint failed**. '산소포화도 250%'는 입력되는 순간 거부됩니다. 5주차에 MIN/MAX로 이상값을 '사후에' 찾았다면, CHECK는 **사전에** 차단합니다 — 둘 다 필요합니다.

ALTER TABLE — 운영 중인 구조 고치기

DDL

```sql
-- 흡연 여부 컬럼 추가 (연구 변수 추가)
ALTER TABLE patient ADD COLUMN smoking TEXT;

-- 컬럼 이름 변경
ALTER TABLE patient RENAME COLUMN smoking TO smoking_status;

-- 테이블 삭제 (구조와 데이터 모두 소멸!)
DROP TABLE vital_demo;
```

> DROP은 되돌릴 수 없습니다. 참조하는 자식 테이블이 있으면 삭제가 거부되는 것도 참조 무결성의 보호막입니다. **운영 DB에서 DROP은 결재 문서와 함께 움직이는 명령**입니다.

테이블 설계 = 연구 설계다

연구 설계

증례기록지(CRF)의 항목이…

- 대상자번호 → `subject_id TEXT PRIMARY KEY`
- 등록일 → `enroll_date DATE NOT NULL`
- 흡연 여부 → `smoking TEXT CHECK(...)`
- 기저 HbA1c → `baseline_a1c REAL`

설계 때 답해야 할 질문

- 이 값이 **없어도 되는가?** → NOT NULL 여부
- 허용되는 값의 **범위·목록**은? → CHECK
- 무엇으로 행을 **특정**하나? → PK
- 기존 데이터와 **어떻게 연결**되나? → FK

> **CRF(Case Report Form, 증례기록지)**: 연구 대상자별 수집 항목을 정의한 표준 양식. CRF의 각 항목이 컬럼으로, 허용값 규칙이 제약조건으로 번역됩니다.

실습 — 내 연구용 코호트 테이블을 직접 설계한다

실습

```sql
CREATE TABLE my_cohort (
  subject_id   TEXT PRIMARY KEY,
  patient_id   TEXT NOT NULL
               REFERENCES patient(patient_id),
  index_date   DATE NOT NULL,        -- 코호트 진입일
  age_at_index INTEGER
               CHECK (age_at_index BETWEEN 0 AND 120),
  sex          TEXT NOT NULL CHECK (sex IN ('M','F')),
  dm_flag      INTEGER NOT NULL DEFAULT 0
               CHECK (dm_flag IN (0,1))  -- 당뇨 동반 여부
);
```

13주차 코호트 실습에서 이 테이블에 실제 데이터를 채워 넣습니다. 오늘의 과제: 각자 관심 주제로 컬럼 6개 이상의 코호트 테이블을 설계해 오세요 — **각 제약조건마다 '왜'를 한 줄씩** 달아서.

이번 주 용어 정리

용어	뜻
DDL (Data Definition Language)	구조를 정의하는 SQL — CREATE·ALTER·DROP
데이터 타입	컬럼이 담을 값의 종류 — INTEGER·REAL·TEXT·DATE
제약조건(constraint)	입력 규칙 — 위반한 문장은 통째로 거부됨
CHECK	허용 값의 조건을 직접 지정 (예: spo2 BETWEEN 0 AND 100)
DEFAULT	값을 생략했을 때 자동으로 들어가는 기본값
REFERENCES	외래키 선언 — 참조할 부모 테이블(컬럼)을 지정
CRF(증례기록지)	연구 수집 항목을 정의한 표준 양식 — 스키마 설계의 원본

7주차 정리

- 타입 선택이 첫 번째 검증: **검사값은 REAL, 날짜는 통일된 DATE**
- 제약조건 5총사(PK · NOT NULL · UNIQUE · CHECK · FK) = **입력 사고의 사전 차단**
- DROP은 비가역 — 참조 무결성이 실수를 한 번은 막아준다
- **CRF → 스키마** 번역 능력이 곧 연구 데이터 품질

과제 '내 연구 코호트 테이블' CREATE TABLE 문 제출 (제약조건 사유 주석 포함)

다음 주 INSERT·UPDATE·DELETE — 그리고 의무기록은 왜 진짜로 지우지 않는가

8주차

INSERT · UPDATE · DELETE 그리고 지우지 않는 기록

- INSERT — 새 기록 넣기
- UPDATE — 오입력 정정 (WHERE를 빼먹으면?)
- DELETE와 soft delete — 의무기록은 왜 진짜로 지우지 않는가
- 트랜잭션과 ACID — 정정 입력 중 정전이 나면

> **이번 주 대표 질문**
>
> 잘못 입력된 검사값을 어떻게 '정정'해야 하는가?

INSERT — 새 환자 등록

INSERT

```sql
-- 컬럼을 명시하고 순서대로 값 입력 (권장)
INSERT INTO patient (patient_id, sex, birth_date, blood_type)
VALUES ('P051', 'F', '1988-03-15', 'A+');

-- 여러 행 한 번에
INSERT INTO patient (patient_id, sex, birth_date, blood_type)
VALUES ('P052', 'M', '1979-11-02', 'O+'),
       ('P053', 'F', '2001-06-21', 'B+');
```

> 컬럼명을 생략하면 '테이블 정의 순서'에 의존하게 되어, 나중에 ALTER로 컬럼이 추가되는 순간 코드가 깨집니다. **항상 컬럼명을 명시**하세요.

INSERT vs 제약조건 — 7주차의 복습

임상 질문

방금 넣은 환자가 규칙(제약조건)을 어기면?

```
INSERT INTO patient
  (patient_id, sex, birth_date, blood_type)
VALUES ('P001', 'X', '1988-03-15', 'A+');
```

두 가지가 동시에 걸립니다: P001은 **PK 중복**, 'X'는 **CHECK(sex IN ('M','F')) 위반**. 잘 설계된 스키마에서는 나쁜 데이터가 입구에서 튕겨 나갑니다.

UPDATE — 조건에 맞는 행만 수정

임상 질문

오입력 정정: 혈당 검사 L0044의 값을 167에서 176으로 (재검 결과 반영)

```
UPDATE lab
SET result_value = 176
WHERE lab_id = 'L0044';

SELECT lab_id, test_name, result_value
FROM lab WHERE lab_id = 'L0044';
```

lab_id	test_name	result_value
L0044	Glucose	176

총 1행

UPDATE의 생명은 **WHERE**입니다. 이 문장은 정확히 한 행만 바꿉니다. 그런데 만약 WHERE를 빼먹는다면…?

WHERE 없는 UPDATE — 전 행이 바뀐다

주의

임상 질문

악몽 시나리오: WHERE 없는 UPDATE

```
UPDATE lab
SET result_value = 176;

SELECT COUNT(*) AS 같은값이된행수
FROM lab WHERE result_value = 176;
```

같은값이된행수
93

총 1행

검사 93건 전체가 176이 되었습니다(이 슬라이드는 사본 DB에서 실행한 결과입니다). 실무 수칙: ① **같은 WHERE로 SELECT를 먼저 실행해 대상 확인** → ② **그 다음 UPDATE**.

의무기록은 왜 진짜로 지우지 않는가

핵심

법과 원칙

- 의료법상 진료기록은 **보존 의무**(기록 종류별 보존연한)
- 정정이 필요하면 **수정 이력이 남는 방식**으로
- "누가, 언제, 무엇을, 왜 바꿨나"가 재구성 가능해야
- 의료분쟁 시 기록의 무결성이 곧 신뢰성

기술적 구현

- **Soft delete**: 행을 지우는 대신 `deleted_at` 시각만 기록
- **감사 로그(audit log)**: 변경 전후 값·수행자·시각을 별도 테이블에 자동 적재
- DELETE 권한 자체를 극소수에게만 (14주차 권한 관리)

> 그래서 병원 DB에서 DELETE는 '기록 말소'가 아니라 **'화면에서 숨김' + '이력 보존'**으로 구현됩니다.

soft delete + 감사로그 — 정정의 올바른 모양

실무 패턴

```sql
-- (1) 검사 정정을 이력으로 남기기
CREATE TABLE lab_audit (
  audit_id   INTEGER PRIMARY KEY,
  lab_id     TEXT, old_value REAL, new_value REAL,
  changed_by TEXT, changed_at DATETIME,
  reason     TEXT                       -- '검체 재검으로 정정'
);

-- (2) 삭제 대신 표시만
ALTER TABLE lab ADD COLUMN deleted_at DATETIME;
UPDATE lab SET deleted_at = '2026-04-02 09:30'
WHERE lab_id = 'L0093';   -- DELETE FROM lab ... 대신!
```

이후 모든 조회에 `WHERE deleted_at IS NULL`을 붙이면 '지워진 것처럼' 보이지만 기록은 남습니다. 실제 EMR의 정정·보존 구조는 이보다 정교합니다 — 원리를 보여주는 교육용 축약 예시입니다.

정정 입력 중 정전이 나면 — 트랜잭션과 ACID

시나리오 — 검사값 정정은 '값 수정 + 이력 기록' 두 문장이 한 묶음:

```
BEGIN TRANSACTION;
UPDATE lab SET result_value = 176
  WHERE lab_id = 'L0044';     -- ① 값 정정
INSERT INTO lab_audit
  (lab_id, old_value, new_value,
   changed_by, changed_at, reason)
VALUES ('L0044', 167, 176, 'S001',
   '2026-04-02 09:30', '재검 반영');  -- ② 이력
COMMIT;
```

두 문장 **사이에서 정전**되면? — 값은 바뀌었는데 이력이 없는, 감사 추적이 끊긴 상태.

트랜잭션은 "둘 다 되거나, 둘 다 안 되거나"를 보장합니다. COMMIT 전에 장애가 나면 전체가 자동 취소(ROLLBACK)됩니다.

ACID — 한 장 요약 (투약 사고의 언어로)

특성	약속	병원 버전
Atomicity 원자성	전부 되거나 전부 안 되거나	정정+이력이 반쪽만 실행되는 일은 없다
Consistency 일관성	규칙을 어긴 상태로 끝나지 않는다	제약조건 위반 상태로 저장 불가
Isolation 독립성	동시 작업이 서로를 침범하지 않는다	외래·병동이 동시에 수정해도 뒤섞이지 않음
Durability 영속성	COMMIT된 것은 장애에도 살아남는다	'저장 완료'된 처방은 정전에도 사라지지 않음

ROLLBACK은 수동 취소도 가능: 대량 UPDATE 전에 `BEGIN`으로 열고, 결과를 확인한 뒤 이상하면 `ROLLBACK` — 실습 노트북에서 직접 해봅니다.

연습문제

실습

- **Q1.** 가상 환자 P051을 INSERT하고, visit에 외래 내원 한 건을 연결해 보라
- **Q2.** P051의 혈액형을 UPDATE로 정정하라 (WHERE 확인 습관 지키기)
- **Q3.** WHERE 없는 UPDATE를 (사본에서) 실행해 보고 피해 규모를 확인하라
- **Q4.** BEGIN → UPDATE → ROLLBACK으로 '없던 일로 만들기'를 실습하라
- **Q5.** lab_audit 테이블에 정정 이력 한 건을 직접 남겨 보라

다음 주는 중간고사(1~8주차) — 범위 요약본이 노트북에 포함되어 있습니다.

이번 주 용어 정리

용어	뜻
DML (Data Manipulation Language)	데이터를 다루는 SQL — SELECT·INSERT·UPDATE·DELETE
트랜잭션(transaction)	전부 되거나 전부 안 되어야 하는 작업 묶음 (BEGIN ~ COMMIT)
COMMIT / ROLLBACK	트랜잭션 확정 / 전체 취소
ACID	원자성·일관성·독립성·영속성 — 트랜잭션이 지키는 네 가지 약속
soft delete	행을 지우는 대신 삭제 시각만 표시 — 조회에서만 숨김
감사로그(audit log)	변경 전후 값·수행자·시각·사유를 남기는 이력 테이블

8주차 정리

- INSERT는 컬럼명 명시, UPDATE·DELETE는 **같은 WHERE로 SELECT 먼저**
- 의무기록은 지우지 않는다 — **soft delete + 감사로그**가 법적 보존 의무의 구현 방식
- 트랜잭션: 묶음 작업은 **전부 아니면 전무** (BEGIN·COMMIT·ROLLBACK)
- ACID 네 글자면 '정전 나면 처방은 어떻게 되나요?'에 답할 수 있다

다음 주 9주차 중간고사 → 10주차 모델링과 정규화 ("왜 처방 테이블에 환자 이름이 없을까?")

10주차

데이터 모델링과 정규화
왜 처방 테이블엔 환자 이름이 없을까

- ERD — 개체 · 속성 · 관계 그리기
- 이상현상(Anomaly)을 임상 사고로 이해하기
- 정규화 1NF → 2NF → 3NF
- 표준 모델 맛보기: OMOP CDM

> **이번 주 대표 질문**
> 왜 처방 테이블에는 환자 이름이 없을까?

모델링 — 현실을 테이블로 옮기는 3단계

- **개념적 설계**

  현실을 개체(Entity)·관계(Relationship)로 추상화 → ERD. "환자가 내원하고, 내원마다 진단이 달린다"

- **논리적 설계**

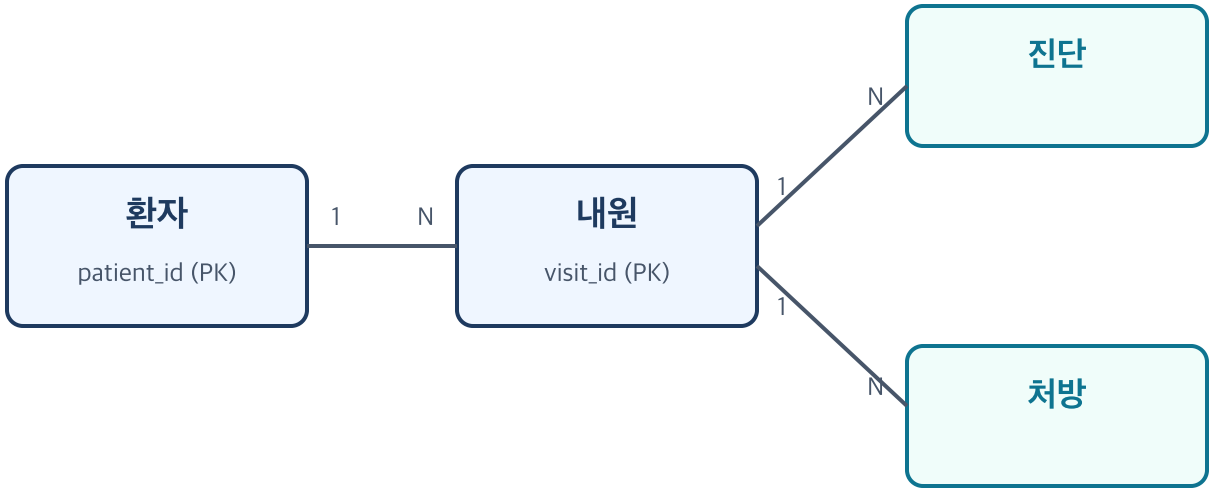
  ERD를 테이블·PK·FK로 변환 — 우리가 6주간 쓴 미니 EMR 스키마가 이 산출물

- **물리적 설계**

  저장 방식·인덱스(조회 가속용 색인)·용량 계획 (DBA의 영역, 이 수업에선 개념만)

개체(entity): 데이터로 관리할 대상(환자·내원·진단) · **관계(relationship)**: 개체 사이의 연결(1:1, 1:N, M:N) · 미니 EMR의 스키마는 이 3단계 중 논리적 설계의 산출물입니다.

ERD 그리기 — 환자·내원·진단 관계 (IE 표기법)



읽는 법

- 사각형 = **개체**(테이블이 될 후보)
- 선 + 1/N = **관계와 참여 수**
- 환자 1명 — 내원 N번 (1:N)
- M:N 관계는 **중간 테이블**로 풀어낸다 (예: 처방 = 내원과 약물의 M:N을 푼 테이블)

만약 전부 한 테이블이었다면 — '통합 진료기록' 가상 사례

문제 상황

내원ID	환자이름	전화번호	진료과	진단명	약품명
V001	김철수	010-1111-2222	내과	고혈압	암로디핀
V002	김철수	010-1111-2222	내과	고혈압, 당뇨	암로디핀, 메트포르민
V003	김철수	010-9999-8888	내과	고혈압	암로디핀
V004	박영희	010-3333-4444	산부인과	정기검진	—

보이시나요? ① V002 한 칸에 진단 두 개(원자성 위반) ② 김철수의 전화번호가 **행마다 복사**되다가 V003만 변경되어 어느 게 진짜인지 모름 ③ 박영희 행을 지우면 **산부인과가 있었다는 사실**도 함께 사라짐

이상현상(Anomaly) — 임상 사고의 언어로

이상현상	정의	임상 시나리오
갱신 이상	중복된 값 중 일부만 수정되어 불일치 발생	개명·전화번호 변경이 100개 행 중 97개만 반영 → 연락 두절, 검사 결과 통보 실패
삽입 이상	불필요한 정보 없이는 입력 자체가 불가	아직 내원 안 한 신규 환자를 등록하려면 가짜 내원 기록이 필요
삭제 이상	행을 지우면 무관한 정보까지 소실	마지막 환자 기록 삭제 시 그 진료과 정보 자체가 사라짐

원인은 하나 — **서로 다른 주제(환자 신상 / 내원 / 진단)가 한 테이블에 동거**하기 때문. 해법이 정규화입니다.

정규화 1NF → 2NF → 3NF (이 수업은 여기까지)

단계	규칙	위반 사례 → 교정
1NF	한 칸엔 하나의 값(원자성)	진단명 칸의 "고혈압, 당뇨" → 진단을 **행으로 분리** (그래서 diagnosis가 별도 테이블)
2NF	복합키의 **일부**에만 매달린 속성 분리	(내원ID, 약품코드) 키인 처방 테이블 안의 '약품명' — 약품코드에만 종속 → 약품 마스터로 분리
3NF	키가 아닌 속성끼리의 종속 분리	내원 테이블 안의 '담당의 전화번호' — 담당의ID에 종속 → staff 테이블로 분리

> 이 수업에서는 **3NF까지** 다룹니다 — 실무·연구 데이터 설계도 대개 이 수준을 사용합니다. BCNF·4NF·5NF는 이론 과목의 영역으로 남겨둡니다.

분리하되, 연결한다

대표 질문

임상 질문

"왜 처방 테이블에 환자 이름이 없을까?" — 정규화의 답

```
SELECT p.rx_id, p.drug_name,
       v.patient_id, v.visit_date
FROM prescription p, visit v
WHERE p.visit_id = v.visit_id
LIMIT 5;
```

rx_id	drug_name	patient_id	visit_date
R0001	암로디핀	P001	2025-11-22
R0002	메트포르민	P001	2025-11-22
R0003	암로디핀	P001	2024-08-16
R0004	메트포르민	P001	2024-08-16
R0005	암로디핀	P001	2023-03-23

총 5행

처방에는 환자 정보가 **없어도 됩니다** — visit_id만 있으면 환자까지 거슬러 올라갈 수 있으니까. 환자 정보는 patient **한 곳에만** 존재 → 개명해도 한 행만 고치면 끝. 같은 사실을 여러 곳에 적지 않는 것 — 정규화의 기본 원칙입니다.

정규화 vs 반정규화 — 목적이 다르면 모양도 다르다

정규화된 운영 DB (EMR)

- 중복 최소, 무결성 최우선
- 입력·수정이 잦은 **진료 현장용**
- 질문 하나에 JOIN 여러 번은 감수

반정규화된 분석 테이블 (연구용)

- 분석 편의를 위해 **의도적으로 중복 허용**
- 환자·진단·검사를 한 판에 펼친 '플랫 테이블'
- CDW·연구 데이터셋이 보통 이 모양

13주차에 여러분이 만들 코호트 테이블이 바로 반정규화된 분석 테이블입니다 — 원본은 정규형으로 지키고, **분석본은 목적에 맞게 펼친다**.

표준 모델 한 장 — OMOP CDM

OMOP CDM이란

- 전 세계 병원이 각자의 EMR을 **같은 구조**로 변환하는 공통 데이터 모델
- person, visit_occurrence, condition_occurrence, drug_exposure, measurement …
- 구조가 같으면 **분석 코드를 병원 간 공유** 가능 → 다기관 연구의 기반

익숙하지 않나요?

- person ≈ patient
- visit_occurrence ≈ visit
- condition_occurrence ≈ diagnosis
- drug_exposure ≈ prescription
- measurement ≈ lab·vital

미니 EMR은 OMOP의 **축소 모형**으로 설계되었습니다.

공식 명칭: **OMOP Common Data Model** — OHDSI(Observational Health Data Sciences and Informatics, 국제 관찰형 보건데이터 컨소시엄)의 표준. 국내 다수 대형병원이 변환을 완료했으며, 공개 문서(ohdsi.org)에서 전체 테이블 정의를 볼 수 있습니다.

연습문제 — 손으로 그리는 과제

실습

- **Q1.** '수술(operation)' 기록을 미니 EMR에 추가한다면 — 개체·속성·관계를 ERD로 그려보라

  어느 테이블과 1:N인가? 집도의는 어떻게 연결하나?

- **Q2.** 위 가상 '통합 진료기록' 테이블을 3NF까지 분해해 보라 (테이블 이름과 PK·FK 명시)

- **Q3.** 우리 prescription 테이블에서 (drug_code, drug_name)이 함께 있는 것은 몇 정규형 관점에서 논쟁거리인가? 실무적으로 왜 남겨두었을까?

  힌트: 2NF, 반정규화

이번 주 용어 정리

용어

용어	뜻
개체(entity) / 관계(relationship)	관리 대상(환자·내원) / 개체 간 연결(1:1 · 1:N · M:N)
ERD	개체·속성·관계를 그린 설계도 — Entity-Relationship Diagram
이상현상(anomaly)	중복 탓에 생기는 갱신·삽입·삭제 사고
함수적 종속	A가 정해지면 B가 유일하게 정해지는 관계 (A → B)
정규화 / 정규형(NF)	중복을 제거하는 분해 절차 / 그 단계 (1NF~3NF)
반정규화	분석 편의를 위해 의도적으로 중복을 되살리는 설계
플랫 테이블	여러 테이블을 한 판에 펼친 분석용 넓은 테이블
OMOP CDM	병원 간 공통 연구용 데이터 모델 (OHDSI 표준)

10주차 정리

- 모델링 3단계: **개념적 설계(ERD) → 논리적 설계(테이블·키) → 물리적 설계(저장·인덱스)**
- 이상현상 = 중복의 대가 — 갱신 누락, 유령 입력, 정보 소실
- 정규화의 원칙: **같은 사실을 여러 곳에 적지 않는다**. 이 수업 범위는 3NF까지
- 운영은 정규형, 분석은 반정규형 — **OMOP CDM**이 두 세계를 잇는 표준

과제 Q1~Q3 (ERD는 손그림 사진 제출 가능)

다음 주 JOIN — 테이블을 정식으로 연결한다 ("당뇨 환자들의 HbA1c를 보려면?")

11주차

JOIN
여러 테이블을 연결해 조회하기

- INNER JOIN — 짝이 있는 것만
- 3개 이상 테이블 연결하기
- LEFT JOIN — 짝이 없어도 남긴다
- LEFT JOIN으로 '검사를 안 받은 환자' 찾기

> **이번 주 대표 질문**
> 당뇨 환자들의 HbA1c 결과를 보려면?

왜 JOIN이 필요한가

개념

10주차에 우리는

- 중복을 없애려 테이블을 **분리**했다
- 환자 정보는 patient에만, 진단은 diagnosis에만

그 대가

- "당뇨 환자의 검사 결과" 같은 질문은 이제 **여러 테이블에 걸쳐** 있다

JOIN이 하는 일

여러 테이블의 관련 행을 **공통 키**를 기준으로, **조회 시점에** 조합해 하나의 결과로 만든다.

저장은 나눠서(무결성), 조회는 붙여서(편의) — 관계형 DB의 핵심 원리입니다.

INNER JOIN 기본형

임상 질문

내원 기록에 환자의 성별·생년월일을 붙여줘

```
SELECT v.visit_id, v.visit_date,
       v.department,
       p.sex, p.birth_date
FROM visit v
JOIN patient p
  ON v.patient_id = p.patient_id
LIMIT 6;
```

visit_id	visit_date	department	sex	birth_date
V0001	2025-11-22	내과	F	1947-04-12
V0002	2024-08-16	내과	F	1947-04-12
V0003	2023-03-23	내과	F	1947-04-12
V0004	2024-10-13	내과	F	1942-01-04
V0005	2025-05-23	내과	F	1942-01-04
V0006	2025-07-21	내과	F	1942-12-27

총 6행

`ON` 이 연결 조건(어떤 열쇠로 잇는가), `v, p` 는 테이블 별칭. 같은 이름의 컬럼이 양쪽에 있으니 **어느 테이블의 컬럼인지 항상 명시**하는 습관을.

다중 JOIN — 대표 질문 풀기

대표 질문

임상 질문

당뇨(E11) 환자들의 HbA1c 결과를 보려면? — 4개 테이블 연결

```
SELECT DISTINCT v.patient_id,
       l.result_value AS hba1c,
       l.collected_at
FROM diagnosis d
JOIN visit v  ON d.visit_id = v.visit_id
JOIN visit v2 ON v2.patient_id = v.patient_id
JOIN lab l    ON l.visit_id = v2.visit_id
WHERE d.kcd_code LIKE 'E11%'
  AND l.test_name = 'HbA1c'
ORDER BY v.patient_id, l.collected_at;
```

patient_id	hba1c	collected_at
P001	8.1	2023-03-23 07:03:00
P001	7.9	2024-08-16 07:42:00
P001	7.6	2025-11-22 07:22:00
P003	6.7	2023-05-30 07:20:00
P003	7.3	2023-12-01 07:14:00
P003	8.5	2025-07-21 07:36:00

… 총 17행 중 6행 표시

진단은 이번 내원에, 검사는 다른 내원에 있을 수 있어 visit을 두 번 통과했습니다(같은 환자의 모든 내원으로 확장). JOIN은 체인처럼 이어 붙이면 됩니다 — **ERD를 보며 열쇠의 경로를 따라가는 것**이 요령.

INNER JOIN vs LEFT JOIN — 짝이 없는 행의 운명

INNER JOIN

- 양쪽 모두에 짝이 있는 행만 생존
- "진단도 있고 검사도 있는" 환자만
- 짝 없는 행은 **조용히 사라진다** — 눈치채지 못하면 위험

LEFT JOIN

- 왼쪽 테이블의 행은 **전부 생존**
- 오른쪽에 짝이 없으면 그 자리는 NULL
- "안 받은 사람"이 NULL로 **보이게 된다**

연구 관점 번역: INNER는 '둘 다 있는 사람 분석', LEFT는 '**없는 사람이 누구인지**까지 분석'. 결측을 다루는 순간 LEFT JOIN이 기본값이 됩니다.

LEFT JOIN 실전 — 전원 유지하며 붙이기

임상 질문

당뇨 환자 전원 명단에 HbA1c 검사 여부를 표시하라

```
SELECT dm.patient_id,
       l.result_value AS hba1c
FROM (SELECT DISTINCT v.patient_id
      FROM diagnosis d
      JOIN visit v ON d.visit_id = v.visit_id
      WHERE d.kcd_code LIKE 'E11%') dm
LEFT JOIN visit v2 ON v2.patient_id =
dm.patient_id
LEFT JOIN lab l ON l.visit_id = v2.visit_id
              AND l.test_name = 'HbA1c'
GROUP BY dm.patient_id;
```

patient_id	hba1c
P001	7.6
P003	8.5
P005	6.9
P007	∅
P010	9.5
P012	7.3
P013	∅
P020	7.2

총 8행

hba1c가 **∅(NULL)인 환자가 보입니다** — INNER JOIN이었다면 명단에서 통째로 사라졌을 사람들. "검사를 안 받았다"는 사실은 LEFT JOIN이어야만 화면에 나타납니다.

결측 환자 찾기 — 질 지표의 원형

대표 질문

임상 질문

그래서, HbA1c 검사를 한 번도 안 받은 당뇨 환자는 누구인가?

```sql
SELECT DISTINCT v.patient_id
FROM diagnosis d
JOIN visit v ON d.visit_id = v.visit_id
WHERE d.kcd_code LIKE 'E11%'
  AND v.patient_id NOT IN (
    SELECT v2.patient_id
    FROM lab l
    JOIN visit v2 ON l.visit_id = v2.visit_id
    WHERE l.test_name = 'HbA1c');
```

patient_id
P007
P013

총 2행

P007, P013 — 당뇨 진단은 있는데 당화혈색소 추적이 없는 환자입니다. 이 쿼리가 **진료 질 지표(당뇨 환자 HbA1c 검사율)**와 **결측 파악**의 원형입니다. 다음 주에 NOT EXISTS로 다시 씁니다.

3테이블 JOIN — 환자 + 내원 + 의료진

임상 질문

진료과별 담당의(staff) 정보까지 — 3테이블 기본 연습

```
SELECT v.visit_id, v.visit_date,
       p.sex AS 환자성별,
       s.role, s.department AS 소속과
FROM visit v
JOIN patient p ON v.patient_id = p.patient_id
JOIN staff s   ON v.attending_id = s.staff_id
WHERE v.visit_type = '입원'
ORDER BY v.visit_date DESC;
```

visit_id	visit_date	환자성별	role	소속과
V0085	2025-08-27	M	의사	내과
V0081	2025-08-10	M	의사	내과
V0088	2025-08-03	F	의사	외과
V0079	2025-05-01	M	의사	신경과
V0063	2025-03-19	F	의사	신경과
V0087	2024-12-06	M	의사	외과

… 총 21행 중 6행 표시

연결 경로: visit→patient는 patient_id로, visit→staff는 attending_id로 — **한 번에 한 열쇠씩**. ON 조건을 빠뜨리면 모든 행의 조합(**카티전 곱**)이 생겨 결과가 폭발하니 주의.

연습문제

실습

- **Q1.** 각 진단(dx_name)에 해당 환자의 성별을 붙여 출력하라

  diagnosis + visit + patient

- **Q2.** 뇌 MRI를 찍은 내원의 환자ID·촬영일·판독문을 출력하라

  imaging + visit

- **Q3.** 영상 검사를 한 번도 안 받은 환자는 몇 명인가?

  힌트: LEFT JOIN … IS NULL 또는 NOT IN

- **Q4.** 처방과 진단을 내원 기준으로 연결해 '폐렴(J18) 내원에서 나간 약물' 목록을 만들라

이번 주 용어 정리

용어	뜻
JOIN	두 테이블을 공통 키로 이어 붙이는 연산
ON 절	어떤 컬럼끼리 연결할지 지정 — 빠지면 모든 조합(카티전 곱) 발생
INNER JOIN	양쪽 모두에 짝이 있는 행만 남김 (기본 JOIN)
LEFT JOIN	왼쪽 테이블은 전부 유지 — 짝 없는 자리는 NULL
테이블 별칭	FROM visit v 의 v — 긴 조인문을 짧고 명확하게
카티전 곱(Cartesian product)	조건 없는 결합 — 100행 × 100행 = 10,000행
∅ (결과 표의 표시)	이 슬라이드 결과 화면에서 NULL을 나타내는 기호

11주차 정리

- JOIN = 정규화로 나눈 테이블을 **열쇠(키)로 조회 시점에 재조립**
- 다중 JOIN은 ERD 위의 **열쇠 경로 따라가기** — 한 번에 한 연결
- INNER는 짝 없는 행을 조용히 버린다 — **결측 연구엔 LEFT JOIN**
- "검사 안 받은 당뇨 환자" 쿼리 = 질 지표·결측 분석의 원형

과제 Colab 11주차 Q1~Q4

다음 주 서브쿼리 — 쿼리 속의 쿼리 ("평균보다 오래 입원한 환자는?", "항생제를 한 번도 안 받은 환자는?")

12주차

서브쿼리
두 단계 질문을 한 문장으로

- 스칼라 서브쿼리 — "평균보다 오래 입원한"
- IN 서브쿼리 — 명단으로 거르기
- EXISTS / NOT EXISTS — "~한 적이 있는가/없는가"
- NOT IN의 NULL 함정

> **이번 주 대표 질문**
> 항생제를 한 번도 처방받지 않은 환자는?

서브쿼리 — 두 단계 질문을 한 문장으로

개념

임상 질문의 실제 구조

"평균보다 오래 입원한 환자는?"

= ① **먼저** 평균 재원일수를 구하고

② 그 값보다 큰 행을 찾아라

SQL의 대응

괄호 안의 쿼리(①)가 **먼저 실행**되어 값·명단이 되고, 바깥 쿼리(②)가 그 결과를 조건으로 사용합니다.

서브쿼리가 놓이는 자리: WHERE(오늘의 중심), FROM(지난주 이미 사용), SELECT

스칼라 서브쿼리 — 값 하나를 조건에 꽂기

대표 질문

임상 질문

평균 재원일수보다 오래 입원한 환자는?

```
SELECT visit_id, patient_id,
       julianday(discharge_date)
         - julianday(visit_date) AS los
FROM visit
WHERE visit_type = '입원'
  AND julianday(discharge_date)
        - julianday(visit_date) >
      (SELECT AVG(julianday(discharge_date)
                - julianday(visit_date))
       FROM visit
       WHERE visit_type = '입원')
ORDER BY los DESC;
```

visit_id	patient_id	los
V0089	P039	35
V0073	P027	20
V0061	P021	19
V0069	P025	19
V0077	P029	17
V0075	P028	14

… 총 7행 중 6행 표시

괄호 안이 먼저 평균(11.7일)을 계산 → 바깥 쿼리가 그보다 긴 입원만 필터. 35일 입원한 P039(대퇴골 골절)가 선두입니다. **기준값을 데이터가 스스로 정하므로** 평균이 바뀌어도 쿼리를 수정할 필요가 없습니다.

IN 서브쿼리 — 명단을 만들어 거르기

임상 질문

뇌경색(I63) 진단을 받은 적 있는 환자의 인적 정보

```sql
SELECT patient_id, sex, birth_date
FROM patient
WHERE patient_id IN (
    SELECT v.patient_id
    FROM visit v
    JOIN diagnosis d
      ON v.visit_id = d.visit_id
    WHERE d.kcd_code LIKE 'I63%');
```

patient_id	sex	birth_date
P021	M	1969-01-02
P022	F	1970-08-24
P023	F	1969-06-14
P024	M	1969-04-28
P025	M	1965-01-15
P026	M	1964-03-16

… 총 10행 중 6행 표시

안쪽 쿼리가 'I63 진단 이력자 명단'을 만들고, 바깥 쿼리가 그 명단에 든 환자만 통과. **"~한 적이 있는 환자군"**은 코호트 정의의 기본 문형입니다(다음 주 주인공).

NOT EXISTS — "~한 적이 전혀 없다"

대표 질문

임상 질문

항생제를 한 번도 처방받지 않은 환자는 몇 명인가?

```sql
SELECT COUNT(*) AS 무항생제환자수
FROM patient p
WHERE NOT EXISTS (
    SELECT 1
    FROM visit v
    JOIN prescription rx
      ON rx.visit_id = v.visit_id
    WHERE v.patient_id = p.patient_id
      AND rx.drug_code IN
          ('CTX2','AMC625','LVX500'));
```

무항생제환자수
41

총 1행

EXISTS는 '한 건이라도 있는가', NOT EXISTS는 '한 건도 없는가'를 묻습니다. 안쪽의 `v.patient_id = p.patient_id` 처럼 바깥 행과 연결된 형태를 **상관 서브쿼리**라 부릅니다 — 환자 한 명 한 명에 대해 질문을 반복하는 그림.

EXISTS의 짝 — 노출군 정의

임상 질문

반대로: 항생제 노출 이력이 있는 환자와 그 약물

```
SELECT DISTINCT v.patient_id,
       rx.drug_name
FROM visit v
JOIN prescription rx
  ON rx.visit_id = v.visit_id
WHERE rx.drug_code IN
      ('CTX2','AMC625','LVX500')
ORDER BY v.patient_id;
```

patient_id	drug_name
P031	세프트리악손 주사
P032	세프트리악손 주사
P033	세프트리악손 주사
P034	세프트리악손 주사
P035	세프트리악손 주사
P036	세프트리악손 주사

… 총 9행 중 6행 표시

약물 노출군 vs 비노출군 — 약물역학 연구의 두 팔이 EXISTS / NOT EXISTS 한 쌍으로 정의됩니다.

NOT IN의 함정 — NULL이 끼면 전부 사라진다

함정

```
-- 위험한 형태
SELECT * FROM patient
WHERE patient_id NOT IN
  (SELECT patient_id FROM some_list);
-- some_list의 patient_id에 NULL이
-- 하나라도 있으면 → 결과 0행!
```

왜?

- `x NOT IN (a, b, NULL)`은 `x≠a AND x≠b AND x≠NULL`
- NULL과의 비교는 참도 거짓도 아닌 **'모름'** → 전체가 '모름' → 아무도 통과 못 함
- 에러도 없이 **조용히 0행** — 최악의 실패 유형

처방

- 결측 가능성이 있으면 **NOT EXISTS를 기본기로**

서브쿼리 vs JOIN — 언제 무엇을

서브쿼리가 자연스러운 질문

- "평균/최댓값 **기준**으로" — 스칼라
- "~인 적 있는/없는 **사람**" — EXISTS류
- 결과에 상대 테이블 컬럼이 **필요 없을 때**

JOIN이 자연스러운 질문

- 양쪽 테이블의 컬럼을 **나란히 출력**해야 할 때
- "진단명과 검사값을 한 줄에"
- 여러 테이블의 속성을 조합한 계산

둘은 상당 부분 서로 변환 가능하며, DBMS가 내부적으로 같은 실행계획으로 바꾸기도 합니다 — SQLite에서는 `EXPLAIN QUERY PLAN` 을 쿼리 앞에 붙이면 실행 방식을 확인할 수 있습니다.

연습문제

실습

- **Q1.** 평균 투약일수보다 길게 처방된 약물 행은?

  스칼라 서브쿼리

- **Q2.** 폐렴(J18) 진단 이력이 있는 환자의 성별·생년월일은?

  IN

- **Q3.** 검사(lab)를 한 번도 받지 않은 환자 수는?

  NOT EXISTS

- **Q4.** 자기 진료과 평균보다 재원이 길었던 입원 건은?

  도전: 상관 서브쿼리

- **Q5.** Q3를 NOT IN으로도 써 보고, 어떤 상황에서 위험해지는지 한 줄로 설명하라

이번 주 용어 정리

용어

용어	뜻
서브쿼리(subquery)	괄호 안에서 먼저 실행되는 내부 쿼리
스칼라 서브쿼리	값 하나를 반환하는 서브쿼리 — 비교 기준으로 사용
상관 서브쿼리	바깥 쿼리의 행을 참조하며 행마다 다시 평가되는 서브쿼리
EXISTS / NOT EXISTS	조건에 맞는 행이 하나라도 있는가 / 전혀 없는가
IN / NOT IN	명단 포함 / 미포함 — NOT IN은 명단에 NULL이 있으면 0행
노출군 / 비노출군	약물 등 연구 요인에 노출된 집단 / 노출되지 않은 대조 집단

12주차 정리

- 서브쿼리 = **두 단계 질문의 한 문장화** — 안쪽이 먼저, 바깥이 나중
- 스칼라(값 하나) · IN(명단) · **EXISTS/NOT EXISTS(이력 유무)**
- "~한 적 없는 환자"는 NOT EXISTS — **NOT IN은 NULL이 섞이면 조용히 0행**이 된다
- 노출군/비노출군 정의 = EXISTS 한 쌍 — 약물역학의 문법

과제 Colab 12주차 Q1~Q5

다음 주 종합 실전 — 연구 질문에서 코호트 추출, 그리고 Table 1까지 한 번에

13주차

임상연구 코호트 추출 실전
연구 질문에서 Table 1까지

- 연구 질문 → 포함·제외 기준 → 쿼리
- Index date와 washout — '첫 진단'의 정의
- 단계별 코호트 조립 (WITH 구문)
- Table 1 자동 생성

> **이번 주 대표 질문**
>
> 2020년 이후 '첫' 뇌졸중 진단 환자를 뽑아보자

연구 질문을 데이터 언어로 번역하기

설계

연구 용어	뜻	SQL 대응물
포함 기준	어떤 환자를 코호트에 넣나	WHERE + JOIN (진단코드, 기간)
제외 기준	누구를 빼나	NOT EXISTS / NOT IN
Index date	각 환자의 '시계 0시' — 추적 시작일	MIN(visit_date) 등으로 정의
Washout	과거 일정 기간 질환 이력이 없어야 '신규'로 인정	HAVING MIN(…) ≥ 기준일
Table 1	코호트의 기저 특성 요약표	CASE × GROUP BY (6주차)

오늘의 연구 질문: **"2020년 1월 1일 이후 처음으로 뇌경색(I63)을 진단받은 성인 환자"**의 코호트를 만들고 기저 특성을 요약한다.

후보군 전체 훑기

임상 질문

1단계 — I63 진단 이력이 있는 환자와 첫 진단일

```sql
SELECT v.patient_id,
       MIN(v.visit_date) AS first_dx_date
FROM visit v
JOIN diagnosis d
  ON v.visit_id = d.visit_id
WHERE d.kcd_code LIKE 'I63%'
GROUP BY v.patient_id
ORDER BY first_dx_date;
```

patient_id	first_dx_date
P030	2019-09-29
P029	2019-11-03
P023	2020-04-21
P021	2020-12-13
P024	2021-03-05
P028	2021-05-25
P025	2023-01-31
P027	2023-03-28
P026	2024-08-05
P022	2025-03-19

총 10행

10명이 잡힙니다. 그런데 P029·P030의 첫 진단은 **2019년** — 이들은 2020년 이후 기준으로는 '신규 발생'이 아니라 '기존 환자'입니다. 여기서 washout이 등장합니다.

Index date 확정

대표 질문

임상 질문

2단계 — washout 적용: '첫' 진단이 2020년 이후인 환자만

```sql
SELECT v.patient_id,
       MIN(v.visit_date) AS index_date
FROM visit v
JOIN diagnosis d
  ON v.visit_id = d.visit_id
WHERE d.kcd_code LIKE 'I63%'
GROUP BY v.patient_id
HAVING MIN(v.visit_date) >= '2020-01-01'
ORDER BY index_date;
```

patient_id	index_date
P023	2020-04-21
P021	2020-12-13
P024	2021-03-05
P028	2021-05-25
P025	2023-01-31
P027	2023-03-28
P026	2024-08-05
P022	2025-03-19

총 8행

HAVING MIN(visit_date) ≥ '2020-01-01' — '가장 이른 진단'이 기준일 이후여야 신규. P029·P030이 제외되어 8명이 남았습니다. 각 환자의 첫 진단일이 곧 **index date**가 됩니다.

3단계 — WITH로 단계를 이름 붙여 조립하기

CTE

```sql
WITH stroke_cohort AS (          -- ① 신규 I63 코호트
  SELECT v.patient_id,
         MIN(v.visit_date) AS index_date
  FROM visit v
  JOIN diagnosis d ON v.visit_id = d.visit_id
  WHERE d.kcd_code LIKE 'I63%'
  GROUP BY v.patient_id
  HAVING MIN(v.visit_date) >= '2020-01-01'
)
SELECT c.patient_id, c.index_date,       -- ② 특성 붙이기
       p.sex,
       CAST(substr(c.index_date,1,4) AS INT)
         - CAST(substr(p.birth_date,1,4) AS INT)
         AS age_at_index
FROM stroke_cohort c
JOIN patient p ON p.patient_id = c.patient_id;
```

WITH(공통 테이블 표현식, CTE): 서브쿼리에 이름을 붙여 단계를 쌓는 문법. 정의서의 순서(포함→제외→특성)가 코드 구조에 그대로 보입니다 — 논문 Methods와 코드가 1:1 대응.

동반질환 플래그 — EXISTS의 재등장

임상 질문

4단계 — 기저 변수 추가: 고혈압(I10) 동반 여부 플래그

```sql
WITH stroke_cohort AS (
  SELECT v.patient_id,
         MIN(v.visit_date) AS index_date
  FROM visit v
  JOIN diagnosis d ON v.visit_id = d.visit_id
  WHERE d.kcd_code LIKE 'I63%'
  GROUP BY v.patient_id
  HAVING MIN(v.visit_date) >= '2020-01-01')
SELECT c.patient_id, c.index_date,
  CASE WHEN EXISTS (
    SELECT 1 FROM visit v2
    JOIN diagnosis d2
      ON d2.visit_id = v2.visit_id
    WHERE v2.patient_id = c.patient_id
      AND d2.kcd_code LIKE 'I10%')
  THEN 1 ELSE 0 END AS htn_comorbid
FROM stroke_cohort c;
```

patient_id	index_date	htn_comorbid
P021	2020-12-13	1
P022	2025-03-19	1
P023	2020-04-21	0
P024	2021-03-05	0
P025	2023-01-31	0
P026	2024-08-05	1
P027	2023-03-28	1
P028	2021-05-25	1

총 8행

EXISTS가 동반질환 플래그 제조기로 재등장 — 같은 패턴으로 어떤 기저 변수도 만들 수 있고, `WHERE NOT EXISTS(...)`로 바꾸면 제외 기준이 된다.

Table 1 생성

Table 1

임상 질문

5단계 — Table 1: 코호트 기저 특성 요약

```
WITH stroke_cohort AS (
  SELECT v.patient_id,
         MIN(v.visit_date) AS index_date
  FROM visit v
  JOIN diagnosis d ON d.visit_id = v.visit_id
  WHERE d.kcd_code LIKE 'I63%'
  GROUP BY v.patient_id
  HAVING MIN(v.visit_date) >= '2020-01-01')
SELECT p.sex AS 성별,
  COUNT(*) AS 명수,
  ROUND(AVG(CAST(substr(c.index_date,1,4) AS INT)
    - CAST(substr(p.birth_date,1,4) AS INT)), 1)
    AS 평균나이
FROM stroke_cohort c
JOIN patient p ON p.patient_id = c.patient_id
GROUP BY p.sex;
```

성별	명수	평균나이
F	4	58.3
M	4	55.3

총 2행

논문 Table 1의 뼈대가 완성됐습니다. 나머지 변수(동반질환율, 약물 노출률)도 같은 GROUP BY에 CASE 플래그를 추가하면 됩니다 — 6·12주차 기술의 총합.

보너스 — 자주 쓰는 코호트는 VIEW로 저장

VIEW

```sql
CREATE VIEW v_stroke_cohort AS
SELECT v.patient_id,
       MIN(v.visit_date) AS index_date
FROM visit v
JOIN diagnosis d ON d.visit_id = v.visit_id
WHERE d.kcd_code LIKE 'I63%'
GROUP BY v.patient_id
HAVING MIN(v.visit_date) >= '2020-01-01';

-- 이후로는 테이블처럼:
SELECT * FROM v_stroke_cohort;
```

VIEW는 **저장된 쿼리**(가상 테이블)입니다. 정의를 한 곳에 두면 공동연구자 모두가 같은 코호트 정의를 쓰게 됩니다 — '나만의 엑셀 사본' 문제(1주차)의 정반대.

조별 프로젝트 안내 — 코호트 정의서 · 추출 쿼리 · Table 1

프로젝트

제출물 (15주차 발표)

- ① **코호트 정의서** 1쪽 — 연구 질문, 포함·제외 기준, index date 정의
- ② **추출 쿼리** — WITH 단계 구조, 단계별 대상자 수 주석
- ③ **Table 1** — 성별·연령 + 조별 선택 변수 2개 이상

평가 기준

- 포함·제외 기준의 **임상적 타당성 (40%)**
- 기준 → 쿼리의 **정확한 번역 (30%)**
- Table 1 해석과 한계 논의 (20%)
- 발표 명료성 (10%)

attrition(단계별 대상자 수 감소 흐름): 포함 → washout → 제외 각 단계의 n을 쿼리 주석으로 남기면 심사와 재현이 쉬워집니다.

이번 주 용어 정리

용어

용어	뜻
포함 / 제외 기준	코호트에 넣을 조건 / 뺄 조건 — 정의서에 글로 먼저 쓴다
index date	환자별 추적 시작일(시계의 0시) — 나이·노출 계산의 기준 시점
washout	과거 일정 기간 이력이 없어야 '신규 발생'으로 인정하는 규칙
CTE (WITH 구문)	서브쿼리에 이름을 붙여 단계로 쌓는 문법 — Common Table Expression
VIEW	저장된 쿼리 — 테이블처럼 조회되는 가상 테이블, 정의 공유용
attrition	단계별 대상자 수 감소 흐름 — 각 단계의 n을 기록
Table 1	코호트의 기저 특성 요약표 — CASE 플래그 × GROUP BY로 생성

13주차 정리

- 코호트 추출 = **포함(JOIN·WHERE) → washout(HAVING MIN) → 제외(NOT EXISTS) → 특성(CASE·EXISTS)**
- Index date가 모든 시간 계산의 원점 — '언제 기준의 나이·노출인가'를 항상 명시
- **WITH(CTE)**로 정의서의 단계를 코드 구조로 — Methods와 1:1 대응
- 공유할 정의는 **VIEW**로 — 연구진 전원이 같은 코호트를 본다

과제 조별 프로젝트 착수 — 다음 주까지 연구 질문·포함 기준 초안

다음 주 이 결과를 믿어도 되나 — 의료데이터의 함정, 가명처리와 IRB, RDB 밖의 데이터

14주차

이 결과를 믿어도 되나?
의료데이터의 함정과 책임

- 단위 불일치 · 측정시각 · 코드 버전 — 3대 함정
- 결측이 유병률을 왜곡하는 방식
- 개인정보보호법 · 가명처리 · IRB
- 영상·파형·판독문 — RDB 밖의 데이터

이번 주 대표 질문

내가 뽑은 이 숫자를 논문에 실어도 되는가?

이 7.8은 저혈당인가?

함정 1

임상 질문

함정 ① 단위 불일치 — 우리 혈당 데이터에 숨어 있던 것

```
SELECT lab_id, test_name,
       result_value, unit
FROM lab
WHERE test_name = 'Glucose'
ORDER BY result_value
LIMIT 5;
```

lab_id	test_name	result_value	unit
L0092	Glucose	7.8	mmol/L
L0089	Glucose	82	mg/dL
L0086	Glucose	101	mg/dL
L0083	Glucose	108	mg/dL
L0027	Glucose	112	mg/dL

총 5행

7.8 mmol/L = 약 140 mg/dL — **단위가 다르면 같은 숫자가 정반대 의미**가 됩니다. 단위 확인 없이 AVG를 돌리면 평균이 통째로 왜곡됩니다. 실무 수칙: 수치 분석 전 `GROUP BY unit`으로 단위 분포부터.

분석 전 30초 감사(audit) 습관

임상 질문

함정 ① 대응 — 단위 감사 쿼리

```sql
SELECT test_name, unit,
       COUNT(*) AS 건수,
       MIN(result_value) AS 최소,
       MAX(result_value) AS 최대
FROM lab
GROUP BY test_name, unit
ORDER BY test_name;
```

test_name	unit	건수	최소	최대
CRP	mg/L	6	48.3	169
Creatinine	mg/dL	4	1.97	2.2
Glucose	mg/dL	27	82	218
Glucose	mmol/L	1	7.8	7.8
HbA1c	%	17	6.7	9.5
Hemoglobin	g/dL	10	9.8	15.7
LDL	mg/dL	28	115	184

총 7행

Glucose에 단위가 **두 종류**인 것이 즉시 드러납니다. 새 데이터셋을 받으면 ①단위 분포 ②MIN/MAX ③결측 수 — 3종 감사부터 실행합니다.

함정 ② 시각의 여러 얼굴 — 어느 시간이 '진짜'인가

같은 검사 하나에도

- **처방 시각** — 의사가 오더 낸 때
- **채혈 시각** (collected_at) — 검체가 몸에서 나온 때
- **결과보고 시각** — 수치가 확정된 때
- 기록 시각 — 시스템에 입력된 때

왜 중요한가

- 공복혈당 연구인데 **오후 채혈**이 섞이면? — 우리 데이터의 함정 검사(14시 채혈)처럼
- '입원 후 48시간 내 항생제'는 **어느 시각 기준**인가에 따라 결과가 달라진다
- 논문 Methods에 시각 정의를 **명시**해야 재현 가능

> 임상적으로 의미 있는 시각은 대부분 **검체 채취·측정 시각**입니다 — 기록 시각과 몇 시간씩 어긋날 수 있습니다.

함정 ③ 코드는 살아 움직인다

- **코드 버전 변경**

  KCD는 주기적으로 개정 — 같은 질환의 코드가 시기별로 다를 수 있다

- **코딩 관행의 변화**

  특정 진단의 '유행'은 질병 증가가 아니라 수가·지침 변화일 수 있다

- **주진단 vs 부진단**

  is_primary를 무시하고 세면 '입원 사유'와 '동반 상병'이 뒤섞인다

- **매핑 테이블의 필요**

  다기관 연구는 코드 → 표준 개념(OMOP) 매핑이 절반의 일

> 진단코드 기반 연구의 고전적 경고: **"코드가 늘어난 것인가, 병이 늘어난 것인가?"**

분모가 다르면 결론이 다르다

임상 질문

결측의 왜곡 — '검사받은 사람만'의 평균이라는 것

```
SELECT
  (SELECT COUNT(DISTINCT v.patient_id)
   FROM visit v JOIN diagnosis d
     ON d.visit_id = v.visit_id
   WHERE d.kcd_code LIKE 'E11%') AS 당뇨환자수,
  (SELECT COUNT(DISTINCT v.patient_id)
   FROM visit v JOIN lab l
     ON l.visit_id = v.visit_id
   WHERE l.test_name = 'HbA1c') AS 검사받은환자
수;
```

당뇨환자수	검사받은환자수
8	6

총 1행

당뇨 환자 8명 중 HbA1c 검사자는 6명 — '평균 HbA1c 7.6%'는 **검사받은 6명의** 평균입니다. 검사를 안 받은 2명(P007·P013)이 조절 양호자인지 추적 탈락자인지에 따라 해석이 뒤집힙니다. 11주차 LEFT JOIN이 이 분모를 지키는 도구였습니다.

개인정보보호 — 연구자가 알아야 할 최소한

법·윤리

구분	내용	예
익명정보	누구인지 복원 불가능 — 개보법 적용 제외	통계표, 완전 집계값
가명정보	추가 정보 없이는 특정 불가하게 처리 과학적 연구 등 목적에 한해 활용 근거 마련(개보법 §28-2 — 요건·예외는 기관 지침 확인)	P001처럼 ID 치환, 생년월일 → 연령대
민감정보	건강정보는 민감정보 — 원칙적 처리 제한, 별도 보호	진단명, 검사결과
IRB	인간대상연구는 기관생명윤리위원회 심의 — 후향적 DB 연구도 예외 아님	심의면제 여부도 IRB가 판단

미니 EMR에 이름·주민번호가 **처음부터 없었던** 이유 — 이 데이터셋 자체가 가명처리의 모형입니다. 표의 내용은 개인정보보호법·생명윤리법의 **2026-07 기준 교육용 요약**이며, 실제 연구 전에는 소속기관 IRB·개인정보보호 담당자와 상의하세요.

접근 권한 — 최소 권한의 원칙 (DCL 한 장)

- **GRANT / REVOKE**

  `GRANT SELECT ON lab TO 연구자;` — 필요한 테이블에, 필요한 동작만

- **역할 기반(RBAC)**

  '연구자' 역할에 조회 권한만 — 개인이 아닌 역할에 권한을 부여

- **감사 추적**

  누가 언제 무엇을 조회했는지 로그 — 8주차 감사로그의 조회 버전

- **연구자의 현실**

  연구자용 CDW 계정은 대부분 SELECT 전용 — 최소 권한 원칙의 적용 결과

> 유출 사고의 다수는 해킹이 아니라 **과도한 권한 + 부주의**에서 시작됩니다.

비정형 텍스트의 한계

비정형

임상 질문

RDB 밖의 데이터 — 판독문은 왜 '검색'이 어려운가

```
SELECT img_id, modality, body_part,
       report
FROM imaging
WHERE report LIKE '%경색%';
```

img_id	modality	body_part	report
IMG001	MRI	뇌	좌측 중대뇌동맥 영역에 급성 뇌경색 소견. 출혈성 변화는 관찰되지 않음.
IMG003	MRI	뇌	좌측 중대뇌동맥 영역에 급성 뇌경색 소견. 출혈성 변화는 관찰되지 않음.
IMG004	MRI	뇌	좌측 중대뇌동맥 영역에 급성 뇌경색 소견. 출혈성 변화는 관찰되지 않음.
IMG005	MRI	뇌	좌측 중대뇌동맥 영역에 급성 뇌경색 소견. 출혈성 변화는 관찰되지 않음.

… 총 8행 중 4행 표시

LIKE 검색은 되지만 — "급성 경색 **소견 없음**"도 '%경색%'에 걸립니다. 부정 표현, 동의어, 오타… 텍스트의 의미는 LIKE로 잡히지 않습니다. 그래서 판독문 연구는 NLP(자연어처리)의 영역으로 넘어갑니다.

영상 · 파형 · 텍스트 — RDB 밖의 저장소

생태계

RDB에 안 넣는 이유

- MRI 한 검사 수백 MB — RDB 내부 저장(BLOB)도 가능하지만 운영·백업·전송 부담이 커진다
- 행·열 구조가 무의미한 픽셀·파형 데이터
- 그래서 **RDB에는 메타데이터**(우리 imaging 테이블), 실제 파일은 PACS·전용 저장소에 두는 구조가 일반적

AI 학습 데이터셋의 구조

- 영상 파일 (PACS) + **레이블** (RDB에서 추출한 진단·판독)
- "폐렴 X-ray 분류"의 **후보 레이블**은 imaging×diagnosis 조합에서 출발 — 시점 정렬·레이블 정의·품질관리는 별도 작업
- SQL은 이 중 **코호트·레이블 후보 추출**에 쓰이는 도구다

> NoSQL·데이터레이크는 이 비정형 세계의 저장 기술입니다 — 관계형이 끝이 아니라, 관계형이 **중심축**인 생태계.

최종 점검 리스트 — "이 결과를 믿어도 되나?"

체크리스트

- **단위** — 같은 검사에 단위가 하나인가? (GROUP BY unit)
- **분모** — 이 평균·비율은 **누구의** 평균·비율인가? (결측·LEFT JOIN)
- **시각** — 어느 시각 기준인가? (채혈 vs 기록)
- **단위 수준** — 내원 수를 환자 수로 착각하지 않았나? (COUNT DISTINCT)
- **코드** — 정의한 코드 패턴이 임상적으로 타당한가? (주진단 여부 포함)
- **권한·윤리** — 이 데이터를 이 목적으로 써도 되는가? (가명처리·IRB)

> 프로젝트 발표(15주차)에서 이 6개 항목으로 서로의 결과를 교차 검증합니다 — 각 항목의 점검 쿼리는 이번 주 노트북에 있습니다.

이번 주 용어 정리

용어	뜻
가명정보	추가 정보 없이는 누구인지 특정 불가하게 처리한 정보 — 연구 목적 활용 근거(개보법 §28-2)
익명정보	복원 자체가 불가능한 정보 — 개인정보보호법 적용 제외
민감정보	건강정보 등 법이 특별히 보호하는 정보 — 원칙적 처리 제한
IRB	기관생명윤리위원회 — 인간대상연구(후향적 DB 연구 포함) 심의 기구
RBAC	역할 기반 접근 제어 — 개인이 아닌 역할(연구자 등)에 권한 부여
감사 쿼리(audit query)	분석 전 단위·범위·결측을 점검하는 사전 검증 쿼리
NLP (자연어처리)	판독문 등 텍스트의 의미를 다루는 기술 — LIKE 검색의 한계 너머
DCL (Data Control Language)	권한을 다루는 SQL — GRANT(부여)·REVOKE(회수)
NoSQL	테이블·SQL 구조를 따르지 않는 DB 총칭 — 문서형·키값형 등, 비정형 대용량에 강점
데이터레이크(data lake)	원시 데이터를 형식 그대로 쌓아두는 대용량 저장소 — 영상·로그·텍스트까지

14주차 정리

- 3대 함정: **단위 · 시각 · 코드** — 분석 전 30초 감사 쿼리가 논문을 지킨다
- 평균·비율 앞에서 항상: **"분모가 누구인가?"**
- 건강정보는 민감정보 — **가명처리 + IRB + 최소 권한**이 연구의 입장권
- 영상·텍스트는 RDB 밖에 있지만, **코호트·레이블 후보 추출에는 SQL**이 쓰인다

과제 프로젝트 최종 점검 — 위 체크리스트 6항목을 자기 결과에 적용해 볼 것

다음 주 15주차 조별 프로젝트 발표 — 코호트 정의서 · 추출 쿼리 · Table 1